Deadline-Aware Task Offloading With Concurrency in Serverless Edge Computing

Minh-Tuong Nguyen, Quang-Trung Luu, *Member, IEEE*, Vo Phi Son, Le-Nam Tran, *Senior Member, IEEE*, and Van-Dinh Nguyen, *Senior Member, IEEE*

***Abstract*—Serverless edge computing enables low-latency Internet of Things (IoT) services but faces scalability challenges due to complex concurrency and resource management. While existing approaches address function initialization and edge–cloud offloading, they often overlook the joint optimization of serverless concurrency and physical-layer resources, leading to potential service degradation and increased costs. To tackle this, we propose `OPLA`, a novel cross-layer framework for joint latency and concurrency optimization, designed to minimize end-to-end latency while optimizing concurrent serverless functions. `OPLA` models interactions between physical-layer resources (e.g., bandwidth, transmission power, offloading ratios) and application-layer concurrency decisions. The formulated problem is a highly nonconvex mixed-integer nonlinear program (MINLP), which we prove to be at least *NP*-complete in certain cases. To approximate its optimal solution efficiently, we propose an iterative exploration–exploitation procedure (EEP). The exploration phase, which is embarrassingly parallelizable, balances solution quality and efficiency with single-parameter tuning. The exploitation phase is just a simple successive convex approximation (SCA) to `OPLA`. Moreover, we also develop a presolve–postsolve heuristic with deterministic rounding to ensure feasibility for `OPLA`. Numerical results demonstrate that EEP consistently achieves solutions within a 6% optimality gap relative to a global solver across a wide range of network scales and workloads, confirming its effectiveness and scalability for real-world serverless edge deployments.**

***Index Terms*—Exploration–exploitation, serverless edge computing, serverless function, task offloading.**

NOMENCLATURE

Abbreviation	Expansion
UE, ES, SCA	User equipment, edge server, and successive convex approximation.

$\otimes, \bigvee, \geqslant, \|\cdot\|_2$	Kronecker product, logical OR, componentwise inequality [37, Section 2.2.4], and Euclidean norm.
$(\cdot)^\top, \mathrm{vec}(\cdot)$	Matrix transpose and vectorization operator.
$\mathcal{CN}(\mu, \sigma^2)$	Complex normal distribution with mean μ and variance σ^2.
$\mathcal{M}, \mathcal{N}, \mathcal{T}$	Sets of UEs, ESs, and ES time slots.
W, K	Total system bandwidth and number of antennas at each ES n.
S_m, C_m	Task size and required CPU cycles for processing task of UE m.
$f_m^{\mathtt{loc}}, f_{mn}^{\mathtt{edg}}$	Local processing capacity of UE m, and allocated processing capacity of ES n to UE m.
$\boldsymbol{h}_{mn}, p_{mn}$	Channel vector, and transmission power from UE m to ES n.
r_{mn}, β_{mn}	Transmission rate and allocated bandwidth portion for UE–ES link.
$\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{X}$	Admission matrix, scheduling matrix, and ES time-slot matrix.
τ_m^{e2e}	Total end-to-end latency of UE m.

Received 11 October 2025; revised 18 January 2026; accepted 9 February 2026. Date of publication 16 February 2026; date of current version 11 May 2026. This work was supported by VUNI.GREEN-X.RISE.AY25-27.03. The work of Quang-Trung Luu was supported by Vietnam Ministry of Education and Training under Grant B2025-BKA-06. The work of Le-Nam Tran was supported by Taighde Éireann—Research Ireland under Grant 22/US/3847 and Grant 13/RC/2077 P2. *(Corresponding author: Van-Dinh Nguyen.)*

Minh-Tuong Nguyen and Vo Phi Son are with the Smart Green Transformation Center (GREEN-X) and the College of Engineering and Computer Science, VinUniversity, Hanoi 100000, Vietnam (e-mail: tuong.nm@vinuni.edu.vn; son.vp@vinuni.edu.vn).

Quang-Trung Luu is with CNRS, CentraleSupélec, L2S, Université Paris-Saclay, 91192 Gif-sur-Yvette, France, and also with the School of Electrical and Electronic Engineering, Hanoi University of Science and Technology, Hanoi 100000, Vietnam (e-mail: quangtrung.luu@centralesupelec.fr).

Le-Nam Tran is with the School of Electrical and Electronic Engineering, University College Dublin, Dublin, D04 V1W8 Ireland (e-mail: nam.tran@ucd.ie).

Van-Dinh Nguyen is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, D02 PN40 Ireland (e-mail: dinh.nguyen@tcd.ie).

Digital Object Identifier 10.1109/JIOT.2026.3665108

I. INTRODUCTION

EDGE computing utilizes distributed computing nodes within the Internet of Things (IoT), typically characterized by limited capacities and positioned near edge servers (ESs), often co-located with wireless access points (APs) [1], [2]. In contrast to cloud computing, which depends on centralized data centers with abundant resources, edge computing operates closer to end users. This proximity reduces latency, alleviates network congestion, and supports latency-sensitive applications. A central technique within this paradigm is task offloading, where computationally intensive tasks are migrated from user devices to ESs to improve processing efficiency and reduce local device workload [3]. However, managing these offloaded tasks effectively requires dynamic and adaptive resource allocation strategies.

Traditional infrastructure-as-a-service (IaaS) models, while effective in cloud environments, struggle in dynamic edge settings due to their dependence on static resource provisioning. Edge nodes, constrained by limited capacity, cannot afford inefficient reservation schemes. Moreover, conventional autoscaling mechanisms often do not respond adequately to rapid workload fluctuations. This mismatch leads to underutilized resources during low demand, as IaaS pricing does not automatically scale efficiently with variable workloads [4].

2327-4662 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

To address the challenges mentioned earlier, function-as-a-service (FaaS) has emerged as a more flexible alternative. In this model, computational tasks are executed as serverless functions that are instantiated dynamically and run on demand [5]. FaaS supports concurrent task offloading from multiple devices while leveraging automatic scaling through an event-driven execution model [6]. Since the introduction of AWS Lambda in 2015, major cloud providers, including Google Cloud Functions, Apache OpenWhisk, and Microsoft Azure, have adopted the serverless paradigm. These systems encapsulate service logic as stateless functions managed by a central orchestrator, which handles invocation, scaling, and resource provisioning automatically [7], [8], [9]. Because functions run in containerized environments, developers are relieved of infrastructure management, allowing them to focus only on application logic. These advantages have driven increasing interest in extending serverless computing to the edge, where its lightweight and scalable nature aligns well with distributed, resource-constrained environments [4], [5], [6].

Despite its promise, serverless edge computing introduces new resource management challenges, particularly in handling concurrent function invocations within elastic autoscaling frameworks [4], [5]. A major concern is resource contention, where multiple concurrent functions compete for limited edge resources, such as CPU, memory, and bandwidth, often resulting in performance bottlenecks [10]. In addition, many serverless platforms impose concurrency limits, restricting the number of simultaneous function executions per node or per function [4], [5]. While these limits help manage resource contention, excessive function instantiation can lead to inefficient resource use and elevated operational costs. Therefore, to fully harness the potential of serverless computing at the edge, efficient scheduling and controlled function scaling mechanisms are critical to balancing performance, cost, and scalability in constrained environments.

A. Related Work

The integration of serverless and edge computing addresses the growing demand for scalable, event-driven execution near data sources, significantly reducing both latency and bandwidth consumption [11], [24]. The work in [12] proposed a cost-minimizing resource procurement framework for hybrid edge–cloud systems. While it did not explicitly use serverless terminology, the framework aligns closely with the on-demand and task-centric nature of serverless computing, dynamically leasing additional resources for edge nodes with limited virtual machines (VMs). Serverless computing simplifies infrastructure management and enables elastic scaling, whereas edge computing enhances decentralization and reduces reliance on cloud resources [15], [16]. However, combining these paradigms introduces challenges in function scheduling, life cycle management, and request distribution, especially under heterogeneous resource availability and network variability [13], [24]. To address these challenges, several system-level strategies have been proposed. For instance, Hu et al. [15] utilized dynamic programming for layer-aware scheduling, leveraging shared image layers to optimize resource usage. From a placement perspective, Shang et al. [16] modeled function placement as an integer programming problem, aiming to balance utilization across edge nodes through capacity-aware allocation strategies.

Task offloading in serverless edge computing is a technique to dynamically offload computational tasks from wireless devices with limited processing capacity to reduce execution latency and avoid energy consumption [19], [20], [21], [22], [25], [26], [27], [28], [29], [30]. Nguyen et al. [19] proposed a decentralized task offloading strategy that reduces latency and outages by dynamically selecting an ES based on queue state and computational capacity. Their focus is on fast outage detection and low-complexity heuristics for server selection, but without joint consideration of radio or compute-layer optimization. Regarding the energy perspective, Zhang et al. [25] extended latency optimization under energy-aware considerations to a multicell environment. The work in [26] incorporated the energy consumption of energy stations, sensor ESs, and users into an optimization problem, which is then solved by using a differential evolution algorithm. Interestingly, the work in [27] tried to align operator revenue with device energy savings by phrasing the problem as a game, which is then addressed using a game-theoretic model. Lin et al. [28] employed a Lyapunov approach to solve the long-term average response latency in a distributed manner using its convex relaxation. Furthermore, end-to-end latency was shown to be reduced by optimizing the user–server admission decision [20]. From a network-wide viewpoint, the work of [29] introduced orthogonal frequency-division multiple access (OFDMA) integrated with spectrum partitioning as an interference-aware solution, jointly optimizing offloading decisions, resource allocation, and computation capacities to effectively reduce energy consumption and manage complexity in ultradense edge environments. Moreover, task offloading in unmanned aerial vehicle nonorthogonal multiple access networks was studied in [21] for latency reduction in augmented or virtual reality applications, while adaptive service installation was investigated to improve latency performance in the recent work of [30]. Beyond energy efficiency, the caching-enhanced offloading technique was shown to reduce execution latency by up to 42.83% for a single-user femto-cloud [22].

Managing concurrency is a critical challenge in serverless edge computing, where simultaneous function invocations can lead to severe resource contention and performance degradation during request bursts [4], [10]. From a fault-tolerance perspective, the work in [31] evaluated a Kubernetes-based serverless architecture and demonstrated that acceptable performance could be maintained up to 90 transactions per second. In response to inefficiencies in concurrent function execution, the work in [32] proposed ProPack, a cross-platform optimization framework that improves performance and reduces operational costs across Amazon, Google, and Azure platforms. Execution environments also play a vital role in concurrency efficiency. WebAssembly (Wasm) was developed in [33] as an alternative to Docker, showing that Wasm-based runtimes significantly reduce memory overhead while supporting higher workload concurrency. For autoscaling optimization, [34] developed analytical models

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF NOTABLE RELATED WORKS

Reference	Problem	Objective	Cross-layer	Approach
[11]	ILP	Latency	✗	SDP relaxation
[12]	ILP	FaaS cost	✗	Ski rental reformulation
[13]	ILP	FaaS cost	✗	GAP reformulation
[14]	QIP	Latency	✗	Randomized rounding
[15]	ILP	FaaS cost	✗	Adaptive DP
[16]	ILP	Latency, FaaS cost, data availability	✗	Shortest path heuristic
[17]	ILP	Latency	✗	GAP heuristic
[18]	ILP	FaaS cost, AoI	✗	Deep RL
[19]	MINLP	Minimum number of outages	✗	Modified Moore-Hodgson
[20], [21]	MINLP	Latency	✗	AO
[22]	ILP	Latency	✗	Coalition game
[23]	Workflow Scheduling	FaaS cost	✗	Gradient
Our work	**MINLP**	**Latency, concurrency invocations**	✓	**Exploration-Exploitation**

AO: Alternating Optimization, AoI: Age of Information, DP: Dynamic Programming, GAP: Generalized Assignment Problem, ILP: Integer Linear Programming, QIP: Quadratic Integer Programming, SDP: Semidefinite Programming

based on concurrency levels and request rates, which were experimentally validated on Knative platforms. In addition, Schuler et al. [35] adopted reinforcement learning (RL) to build adaptive autoscaling policies, demonstrating superior performance over Kubernetes defaults under synthetic workloads using a simple reward mechanism. Although not framed explicitly as a concurrency solution, Pan et al. [23] addressed workflow-level concurrency by optimizing branching structures within serverless workflows. Their approach used gradient-based prewarming and critical path analysis to ensure efficient provisioning of concurrently executing functions.

B. Motivation and Contributions

While prior studies have explored edge service ratio [19], serverless function initialization [14], [15], transmission latency [13], [23], and service costs in edge–cloud frameworks [11], [12], they largely overlooked concurrency management from a cross-layer resource allocation perspective. As summarized in Table I, there is a significant research gap in jointly considering physical-layer radio and computational resources alongside application-layer concurrency control in serverless edge environments. This gap is critical, as managing concurrency levels is essential for ensuring service availability under dynamic demand and constrained resources [4]. Overprovisioning concurrency instances can lead to high operational costs, while underprovisioning can degrade service performance. Hence, effective cross-layer resource allocation strategies are required to meet performance objectives (e.g., latency or task completion constraints) without incurring unnecessary costs.

To address this gap, we propose a novel cross-layer optimization framework, called optimization of latency and concurrency (OPLA). OPLA aims to jointly minimize end-to-end latency and control serverless concurrency levels by optimizing both physical-layer parameters, such as bandwidth, transmission power, admission control, offloading ratios, and processing capacities, and application-layer decisions such as concurrency instances provisioning. This joint optimization enables more efficient and scalable serverless edge computing under realistic and dynamic network conditions.

Summing up, the contributions of this work are as follows.

1) We propose OPLA, a novel cross-layer resource allocation framework that minimizes end-to-end latency of users' offloadings by jointly optimizing physical-layer resources (e.g., bandwidth, transmission power, offloading ratios, and processing capacities) and application-layer concurrency control. The proposed framework also includes user–server admission control as part of the optimization process, which results in a nonconvex mixed-integer nonlinear program (MINLP).
2) To address the intractability of the MINLP, we propose an iterative *exploration–exploitation* approach. The exploration phase, designed for parallel execution, requires single-parameter tuning and uses an exact-penalty successive convex approximation (SCA) to efficiently search the solution space. The exploitation phase then applies a deterministic rounding rule followed by a simplified SCA. A *presolve–postsolve* routine is also embedded to ensure feasibility. This framework leverages hidden convex structures via the SCA principle for a tractable reformulation, following [36].
3) We evaluate the proposed OPLA framework under a wide range of workload and resource conditions. Numerical results demonstrate that our method consistently outperforms benchmark schemes while remaining within approximately 6% of the global optimal solution.

C. Paper Structure and Notations

The remainder of this article is organized as follows. Section II introduces the system model. Section III presents the problem formulation. In Section IV, we detail the proposed exploration–exploitation approach for solving the OPLA problem. Section V reports simulation results to validate the effectiveness of OPLA. Finally, Section VI concludes this

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

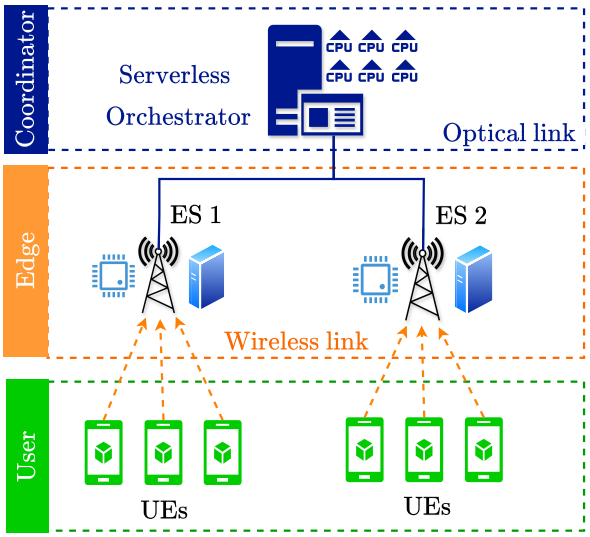


Fig. 1. Illustration of the hierarchical computing network.

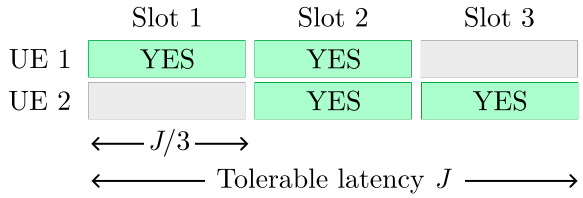


Fig. 2. Illustration of allocating three time slots to 2 UEs.

article. Nomenclature summarizes the main abbreviations and notations used throughout this article.

II. System Model

Fig. 1 illustrates the hierarchical computing network under consideration, which comprises a serverless orchestrator and a set $\mathcal{N}$ of N ESs that serve a set $\mathcal{M}$ of M UEs. The proposed system model is also applicable to broader industrial scenarios such as factory automation, where various IoT devices (e.g., sensors, manipulators, and control units) can also be regarded as UEs. The serverless orchestrator, equipped with the most powerful computing resources, is responsible for making resource allocation decisions [24]. Each ES $n \in \mathcal{N}$ is co-located with an AP equipped with K antennas, enabling wireless connectivity for its associated UEs. We assume that the coordination of ESs is dictated by the serverless orchestrator, and that all ESs operate on a shared system bandwidth under an orthogonal multiple access scheme, i.e., frequency reuse across ESs is not considered. In addition, all ESs are interconnected via high-speed optical links [38], and their coordination follows a master–worker topology [39], as illustrated in Fig. 1. Due to centralized control and low-latency interconnects, it is practically reasonable to assume that coordination delays among ESs are negligible.

A. Channel Model

To model admission control, we define the admission matrix $\boldsymbol{A}_{M\times N}$, where each element a_{mn} is a binary variable indicating whether UE $m \in \mathcal{M}$ is assigned to ES $n \in \mathcal{N}$ ($a_{mn} = 1$) or not ($a_{mn} = 0$). The channel vector between UE m and ES n is expressed as $\boldsymbol{h}_{\mathrm{mn}} = \sqrt{\varphi_{\mathrm{mn}}}\,\overline{\boldsymbol{h}}_{\mathrm{mn}} \in \mathbb{C}^{K\times 1}$, where φ_{mn} denotes the large-scale fading coefficient (e.g., path loss and shadowing), and $\overline{\boldsymbol{h}}_{\mathrm{mn}} \sim \mathcal{CN}(0, \boldsymbol{I}_K)$ represents small-scale Rayleigh fading with independent and identically distributed entries across the K antennas of ES n. The system employs the frequency-division multiple access (FDMA) to allocate bandwidth among UEs. In particular, each admitted UE is assigned a bandwidth share of $\beta_{\mathrm{mn}} W$, where $\beta_{\mathrm{mn}} \in [0, 1]$ represents the fraction of the total system bandwidth W allocated to UE m by ES n. Having this, the transmission rate under FDMA with maximum ratio is given by

$$r_{\mathrm{mn}}(\beta_{\mathrm{mn}}, p_{\mathrm{mn}}) = \beta_{\mathrm{mn}} W \log_2\left(1 + \frac{p_{\mathrm{mn}}\|\boldsymbol{h}_{\mathrm{mn}}\|_2^2}{\beta_{\mathrm{mn}} W N_0}\right) \tag{1}$$

where N_0 is the noise power spectral density.

B. Concurrency Invocations Modeling

Demand scheduling is modeled as a binary scheduling matrix $\boldsymbol{B}_{M\times T}$. Each column of $\boldsymbol{B}$ indicates the potential time slot $t \in \mathcal{T} \triangleq \{1, 2, \ldots, T\}$ allocated for servicing a user's offloading request concurrently. In this context, each time slot refers to a concurrent execution (or concurrent call) for processing these requests. Here, T refers to the maximum number of time slots on each ES. We assume that each offloaded task is executed and completed within a single scheduling slot. This modeling choice is consistent with existing serverless and edge computing frameworks [13], [40], [41], which favor atomic task execution to simplify timing coordination, reduce scheduling complexity, and maintain compatibility with slot-based resource allocation policies. Moreover, such a granularity assumption aligns with low-latency microservice workloads commonly deployed at the edge, such as inference, caching, and control loop applications. Therefore, the final "1" in each row of $\boldsymbol{B}$ denotes the last acceptable execution timestamp for each UE as shown in Fig. 2. For instance, the first UE characterized by the first row of $\boldsymbol{B}$ might tolerate the offloading procedure to occur at the first or second time slot but not beyond. In particular, the first row of $\boldsymbol{B} = \left[\begin{smallmatrix}1&1&0\\0&1&1\end{smallmatrix}\right]$ exemplifies the aforementioned scenario as shown in Fig. 2. Note that the binary matrix $\boldsymbol{B}$ is treated as a parameter that is randomly generated, and we consider that all of its rows must have at least one "1."

When a task offloading decision is executed at time slot t, the orchestrator allocates concurrency instances to handle user requests concurrently [4]. Therefore, designing an efficient offloading scheme that minimizes the number of used time slots, while still meeting all user resource requirements, helps reduce over-provisioning. This, in turn, frees up unused concurrency instances to process additional tasks. An illustrative example is shown in Fig. 2, where both UEs begin task offloading at the start of the time slot 2. Here, J represents the maximum latency tolerance per ES, and J/T denotes the duration of a single time slot.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

Let x_{nt} be a binary decision variable indicating whether a task offloading operation is triggered at the beginning of time slot $t \in \mathcal{T}$ on ES n. It is defined as

$$x_{nt} = \begin{cases} 1, & \text{executing at time slot} t \text{ at ES } n \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

By $\boldsymbol{x}_n \triangleq \left[x_{n1}; \cdots; x_{nT}\right] \in \{0,1\}^{T\times 1}$, the matrix $\boldsymbol{X} \in \{0,1\}^{T\times N}$ is formed by horizontally concatenating the vectors for all N ESs

$$\boldsymbol{X} = [\boldsymbol{x}_1,\ \boldsymbol{x}_2,\ \ldots,\ \boldsymbol{x}_N] \in \{0,1\}^{T\times N}. \tag{3}$$

Denote by $\boldsymbol{X}' \triangleq \mathrm{vec}(\boldsymbol{X})$ the vectorized form of $\boldsymbol{X}$. The variable x_{nt} can then be accessed at index $(n-1)T + t$ in $\boldsymbol{X}'$.

C. Latency Modeling

For a user–server link, the total end-to-end latency for partial task offloading from UE m to ES n, denoted as $\tau_m^{\mathtt{e2e}}$, is modeled as the sum of three components: effective local processing time ($\tau_m^{\mathtt{loc}}$), effective edge processing time ($\tau_{\mathrm{mn}}^{\mathtt{edg}}$), and wireless upload latency ($\tau_{\mathrm{mn}}^{\mathtt{wls}}$) [20], [28]. This relationship is expressed as

$$\tau_m^{\mathtt{e2e}} = \tau_m^{\mathtt{loc}} + \sum_{n\in\mathcal{N}} (\tau_{\mathrm{mn}}^{\mathtt{edg}} + \tau_{\mathrm{mn}}^{\mathtt{wls}}) \tag{4}$$

where $\tau_m^{\mathtt{loc}}$, $\tau_{\mathrm{mn}}^{\mathtt{edg}}$, and $\tau_{\mathrm{mn}}^{\mathtt{wls}}$ are given as

$$\tau_m^{\mathtt{loc}} \triangleq \frac{\left(1 - \sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}}\right) S_m C_m}{f_m^{\mathtt{loc}}} \tag{5a}$$

$$\tau_{\mathrm{mn}}^{\mathtt{edg}} \triangleq \frac{a_{\mathrm{mn}} \lambda_{\mathrm{mn}} S_m C_m}{f_{\mathrm{mn}}^{\mathtt{edg}}} \tag{5b}$$

$$\tau_{\mathrm{mn}}^{\mathtt{wls}} \triangleq \frac{a_{\mathrm{mn}} \lambda_{\mathrm{mn}} S_m}{r_{\mathrm{mn}}(\beta_{\mathrm{mn}}, p_{\mathrm{mn}})} \tag{5c}$$

where S_m represents the task size in bits, and C_m denotes the number of CPU cycles required per bit. The variables $f_m^{\mathtt{loc}}$ and $f_{\mathrm{mn}}^{\mathtt{edg}}$ are the processing capacities of UE m and the allocated ES n resources, respectively. In line with [28] and [20], we assume that ESs generate small response payloads, such as control feedback, classification labels, or scalar computation results. Due to the typically asymmetric bandwidth configuration in edge deployments (i.e., significantly higher downlink than uplink capacity), and the negligible size of these response packets, we exclude downlink transmission delay from the overall wireless latency term, $\tau_{\mathrm{mn}}^{\mathrm{wls}}$. This assumption allows the model to focus on the dominant latency components without sacrificing analytical tractability.

III. Problem Formulation

In this section, we formally define the optimization problem that governs joint user admission, task offloading, resource allocation, and scheduling in the edge–cloud continuum. The goal is to minimize the overall end-to-end sum latency experienced by UEs while minimizing the number of concurrent serverless instances, thereby optimizing both user experience and system efficiency.

A. Constraints

1) ES Placement Constraint: To conserve UE battery life and reduce signaling overhead, each UE $m \in \mathcal{M}$ is allowed to connect to at most one ES $n \in \mathcal{N}$ at a time, i.e.,

$$\sum_{n\in\mathcal{N}} (a)_{\mathrm{mn}} \le 1, \quad \forall m \in \mathcal{M}. \tag{6}$$

2) Scheduling and User Admission Constraint: Scheduling decisions must align with user admission. That is, a user's task can only be scheduled at an ES to which they are admitted

$$\boldsymbol{B}\boldsymbol{X} \geqslant \boldsymbol{A} \tag{7}$$

where the inequality is componentwise [37]. This captures an extended set-cover requirement, ensuring alignment between admission and task execution slots. A symmetric version used in complexity analysis is shown in Appendix A.

3) Bandwidth Constraints: The following constraints ensure that only admitted UEs are allocated bandwidth [constraint (8a)], and the total allocated bandwidth across all UEs does not exceed the available system bandwidth [constraint (8b)]

$$\beta_{\mathrm{mn}} \le a_{\mathrm{mn}}, \ \forall m \in \mathcal{M}, n \in \mathcal{N} \tag{8a}$$

$$\sum_{m\in\mathcal{M}} \sum_{n\in\mathcal{N}} \beta_{\mathrm{mn}} \le 1. \tag{8b}$$

4) Power Constraint: The transmit power that each UE $m \in \mathcal{M}$ allocates to the communication with an ES $n \in \mathcal{N}$ must not exceed its total power budget $P_m^{\max}$

$$p_{\mathrm{mn}} \le a_{\mathrm{mn}} P_m^{\max}, \quad \forall m \in \mathcal{M}, n \in \mathcal{N}. \tag{9}$$

5) Local Processing Constraint: The processing capacity that each UE $m \in \mathcal{M}$ allocates to execute tasks locally $f_m^{\mathtt{loc}}$ is limited by its maximum processing capacity $F_m^{\max}$, i.e.,

$$f_m^{\mathtt{loc}} \le F_m^{\max}, \quad \forall m \in \mathcal{M}. \tag{10}$$

6) ES Processing Capacity Constraint: Similar to (10), the total processing capacity that each ES $n \in \mathcal{N}$ allocates to UEs must not exceed its maximum processing capacity $F_n^{\max}$, i.e.,

$$\sum_{m\in\mathcal{M}} f_{\mathrm{mn}}^{\mathtt{edg}} \le F_n^{\max}, \quad \forall n \in \mathcal{N} \tag{11a}$$

$$f_{\mathrm{mn}}^{\mathtt{edg}} \le a_{\mathrm{mn}} F_n^{\max}, \quad \forall m \in \mathcal{M}, n \in \mathcal{N}. \tag{11b}$$

7) Offloading Server Selection Constraint: To avoid fragmentation and simplify offloading logic, each UE $m \in \mathcal{M}$ can only offload to a single ES $n \in \mathcal{N}$ at a time

$$\sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}} \le 1, \quad \forall m \in \mathcal{M} \tag{12a}$$

$$\lambda_{\mathrm{mn}} \le a_{\mathrm{mn}}, \quad \forall m \in \mathcal{M}, n \in \mathcal{N}. \tag{12b}$$

8) Completion Latency Constraint: Given the total latency parameter J and the division of time into T discrete slots, the start and end times of each slot are defined as $(t-1)\frac{J}{T}$ and $t\frac{J}{T}$, respectively. As discussed in Section II-B, the normalized edge processing time $\frac{S_m C_m}{f_{\mathrm{mn}}^{\mathtt{edg}}}$ for each connected UE is assumed to fall entirely within a single time slot. Accordingly, the latency

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

incurred for processing a task offloaded to a selected ES must satisfy the following disjunctive constraint:

$$\bigvee_{t=1}^{T}\left[\frac{(t-1)J}{T}\le\frac{S_mC_m}{f_{\mathrm{mn}}^{\mathtt{edg}}}\le\frac{tJ}{T}\right],\quad\forall m\in\mathcal{M},n\in\mathcal{N},t\in\mathcal{T}:x_{nt}=1,a_{\mathrm{mn}}=1. \tag{13}$$

In scenarios where task execution exceeds a single time slot, the OPLA framework can be extended by first solving a user admission subproblem. We introduce a binary indicator variable y_i into constraint (13), where $y_i = 1$ if the constraint is satisfied, and $y_i = 0$ otherwise. This leads to a new objective: $\max_{y_i\in\{0,1\}}\sum y_i$, which seeks to maximize the number of users that can be feasibly scheduled. Once the maximal feasible subset is identified, the proposed algorithm in Section IV can be applied to solve the reduced OPLA problem. Remaining users can be handled in subsequent rounds via round-robin or deferral strategies. Since our primary focus is on latency-aware resource scaling under feasible conditions, a full exploration of infeasible scenarios is left for future work. As demonstrated in [42], analyzing system behavior under such constraints presents a promising research direction that warrants dedicated study.

B. Final Formulation

We define the following set of decision variables: $\boldsymbol{\lambda}\triangleq\{\lambda_{\mathrm{mn}}\}_{\forall m,n}$, $\boldsymbol{f}\triangleq\{f_m^{\mathtt{loc}},f_{\mathrm{mn}}^{\mathtt{edg}}\}_{\forall m,n}$, $\boldsymbol{a}\triangleq\{a_{\mathrm{mn}}\}_{\forall m,n}$, $\boldsymbol{\beta}\triangleq\{\beta_{\mathrm{mn}}\}_{\forall m,n}$, $\boldsymbol{p}\triangleq\{p_{\mathrm{mn}}\}_{\forall m,n}$, $\boldsymbol{x}\triangleq\{x_{nt}\}_{\forall n,t}$, and $\boldsymbol{v}\triangleq\{v_{\mathrm{mn}}\}_{\forall m,n}$. Using these variables and the constraints introduced previously, we now formulate the OPLA problem, which aims to minimize both the sum end-to-end latency over all links and the number of active concurrency instances

$$\mathtt{OPLA}:\min_{\substack{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{a}\\ \boldsymbol{\beta},\boldsymbol{p},\boldsymbol{x}}}\ \sum_{m\in\mathcal{M}}\tau_m^{\mathtt{e2e}} \tag{14a}$$

$$\min_{\substack{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{a}\\ \boldsymbol{\beta},\boldsymbol{p},\boldsymbol{x}}}\ \sum_{n\in\mathcal{N}}\sum_{t\in\mathcal{T}}x_{nt} \tag{14b}$$

$$\text{s.t.}\quad (6),(7),(8),(9),(10),(11),(12),(13) \tag{14c}$$

$$\tau_m^{\mathtt{e2e}}\le\tau_m^{\max},\ \forall m\in\mathcal{M} \tag{14d}$$

$$a_{\mathrm{mn}},x_{nt}\in\{0,1\},\ \forall m\in\mathcal{M},n\in\mathcal{N},t\in\mathcal{T} \tag{14e}$$

where $\tau_m^{\max}$ is the maximum permissible end-to-end latency for a user–server link from UE m to ES n.

Theorem 1: OPLA is at least *NP*-complete when UEs offload tasks to their connected ESs.

Proof: See Appendix A.■

It is clear that solving the multiobjective, disjunctive, and mixed-integer nonlinear program OPLA is generally *NP*-hard [43], and may even be undecidable in certain cases [44]. Furthermore, the multiobjective formulation is nonconvex and exhibits a highly combinatorial structure, making it difficult to even determine a weak Pareto solution [11], [45]. The nonconvexity of the problem arises from its highly combinatorial structure and the presence of nonconvex constraints, including logarithmic expressions in the transmission rate. These features render OPLA intractable for standard convex optimization solvers. For these reasons, we propose an approximation method, called *exploration–exploitation*.

IV. Exploration–Exploitation Approach

One common strategy to approximate optimal solutions for multiobjective problems is to employ the weighted sum method, which aims to generate locally Pareto-optimal solutions [11], [45, Section 2.4.4]. However, as noted in [45, Section 3.1], this method requires prior knowledge or accurate estimates of the minimum and maximum values of each objective to compute appropriate scaling factors. These factors are crucial for balancing the objectives when their units or ranges differ significantly.

Algorithm 1 Overall EEP for Solving OPLA (15).

Input: $\tau_{\max}$, $\rho_{\mathtt{step}}$, $\rho_{\max}$, T, J, $\boldsymbol{B}$,$F_{\mathtt{loc}}^{\max}$, $F_n^{\max}$, S_m, C_m, and $P_m^{\max}$, $\forall m,n,t$

Output: Optimal solution ($\boldsymbol{\lambda}^\star$, $\boldsymbol{f}^\star$, $\boldsymbol{a}^\star$, $\boldsymbol{\beta}^\star$, $\boldsymbol{p}^\star$, $\boldsymbol{x}^\star$)

$\boldsymbol{X}'\leftarrow$ Presolve($\boldsymbol{B}_{M\times T}$); ▷Call Function 1 to presolve x_{nt}
Exploration phase
for $\rho=0:\rho_{\mathtt{step}}:\rho_{\max}$ **do**
Call Function 2 to obtain the optimal solution $\boldsymbol{\Psi}_1^\star$
$\boldsymbol{\Psi}_1^\star=$ SCAFirstStage($\bar{\boldsymbol{\Psi}}_1$);
Record ρ and $\boldsymbol{\Psi}_1^\star$ for each time Function 2 is called;
end for
Exploitation phase
Choose $\rho^\star$ and $\boldsymbol{\Psi}_1^\star$ corresponding to the best objective of Function 2;
Pick $\boldsymbol{A}_{M\times N}$ *corresponding to the best objective value* $L(\boldsymbol{\Psi}_1^\star)$ *of Function 2 and round it using Function 3*
$\boldsymbol{A}_{\mathtt{rounded}}=$ RoundingProcedure($\boldsymbol{A}_{M\times N}$);
Call Function 4 to obtain the fine-grained solution $\boldsymbol{\Psi}_2^\star$
$\boldsymbol{\Psi}_2^\star=$ SCASecondStage($\bar{\boldsymbol{\Psi}}_2$);
Call Function 5 to set $x_{nt}=0$ *on disconnected ESs;*
$\boldsymbol{X}'=$ Postsolve($\boldsymbol{A}_{\mathtt{rounded}},\boldsymbol{X}'$);

Function 1 Presolve$(\boldsymbol{B}_{M\times T})$// Presolving $x\triangleq\{x_{nt}\}_{\forall n,t}$

Input: $\boldsymbol{B}_{M\times T}$

Output: Presolve solution $\boldsymbol{X}'$

Create a zero column vector $\boldsymbol{X}'$ of size vec($\boldsymbol{X}$);
[numRows, numCols] = size($I_N\otimes\boldsymbol{B}_{M\times T}$);
colIndices = zeros(numRows, 1);
for $i=1$ **to** numRows **do**;
for $j=1$ **to** numCols **do**
if $A[i,j]==1$ **then**
colIndices[i] = j;
break;
end if
end for
end for
$\boldsymbol{X}'$[colIndices] $\leftarrow 1$;

In the case of the OPLA problem (14), estimating the bounds for the sum end-to-end latency (14a) is particularly challenging due to its nonlinear nature. For the second objective (14b), an upper bound can be derived, as the total number of possible concurrent instances across all ESs is NT. To overcome the limitations of the weighted sum approach, we adopt the ϵ-constraint technique as recommended in [45, Section 3.2.3].

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

Function 2 SCAFirstStage($\bar{\boldsymbol{\Psi}}$)// Solving Problem (20)

Input: An initial point $\bar{\boldsymbol{\Psi}}_1 = \{\bar{\lambda}_{\text{mn}}^{(0)}, \bar{a}_{\text{mn}}^{(0)}, \bar{f}_m^{\text{loc},(0)}\}_{\forall m,n}$
Output: Optimal solution $\boldsymbol{\Psi}_1^\star = (\boldsymbol{\lambda}^\star, \boldsymbol{f}^\star, \boldsymbol{a}^\star, \boldsymbol{\beta}^\star, \boldsymbol{p}^\star)$

$i = 0$;
while $i < I_{\max}$ **do**
$\boldsymbol{\Psi}_1^{(i+1)} = \underset{\boldsymbol{\Psi}_1 \in \mathcal{G}}{\text{argmin}} \sum_{m\in\mathcal{M}} \tau_m^{\text{e2e},(i)} + \rho\omega^{(i)}$;
$\mathcal{G}$ is the set of constraints in problem (27)
if $\|L(\boldsymbol{\Psi}_1^{(i+1)}) - L(\boldsymbol{\Psi}_1^{(i)})\| \le 10^{-2}$ **then**
break;
end if
$i = i + 1$;
end while
Update $\boldsymbol{\Psi}_1^\star = \boldsymbol{\Psi}_1^{(i+1)}$.

Function 3 RoundingProcedure ($A_{M\times N}$)

Input: A real-valued matrix $\boldsymbol{A} \in [0,1]^{M\times N}$
Output: $\boldsymbol{A}_{\texttt{rounded}}$

$\boldsymbol{A}^* = \lfloor \boldsymbol{A} + 1/2 \rfloor$;
if $\boldsymbol{A}^*$ is a non-zero binary matrix **then**
$\boldsymbol{A}_{\texttt{rounded}} = \boldsymbol{A}^*$;
else
if $\boldsymbol{A}^*$ is an all-zero matrix **then**
Create an all-zero $M \times N$ matrix $\boldsymbol{A}_{\texttt{rounded}}$;
for $i = 1$ to M **do**
row = $\boldsymbol{A}[i,:]$;
LB = $1/N - \texttt{tolerance}$;
UB = $1/N + \texttt{tolerance}$;
validIdx = $\{j | \text{row}[j] \in [\texttt{LB}, \texttt{UB}]\}$;
if validIdx $\neq \emptyset$ **then**
maxIdx = $\arg\max_{j\in\texttt{validIdx}} \text{row}[j]$;
$\boldsymbol{A}_{\texttt{rounded}}[i, \texttt{maxIdx}] = 1$;
end if
end for
end if
end if

Function 4 SCASecondStage($\boldsymbol{\Psi}$)

Input: An initial point $\bar{\boldsymbol{\Psi}}_2 = \boldsymbol{\Psi}_2^\star$ obtained from Function 2
Output: Optimal solution $\boldsymbol{\Psi}_2^\star = (\boldsymbol{\lambda}^\star, \boldsymbol{f}^\star, \boldsymbol{\beta}^\star, \boldsymbol{p}^\star)$

Set $i = 0$
while $i < I_{\max}$ **do**
$\mathcal{G}'$ is the set of constraints in problem (30)
$\boldsymbol{\Psi}_2^{(i+1)} = \underset{\boldsymbol{\Psi}_2 \in \mathcal{G}'}{\text{argmin}} \sum_{m\in\mathcal{M}} \hat{\tau}_m^{\text{e2e},(i)}$;
if $\|L(\boldsymbol{\Psi}_2^{(i+1)}) - L(\boldsymbol{\Psi}_2^{(i)})\| \le 10^{-2}$ **then**
break
end if
Set $i = i + 1$;
end while
Update $\boldsymbol{\Psi}_2^\star = \boldsymbol{\Psi}_2^{(i+1)}$.

Function 5 POSTSOLVE(A_{rounded} , X')

Input: UE–ES assignment $\boldsymbol{a}$ from Function 2 and a presolve $\boldsymbol{X}'$
Output: A postsolve $\boldsymbol{X}'$

if $a_{\text{mn}} = 0$ **then**
for each timeslot on disconnected ES **do**
$\boldsymbol{X}'[(n-1)T + t] \leftarrow 0$;
end for
end if

This technique reformulates OPLA into a single-objective problem by treating the first objective as the primary target while constraining (14b) within the total number of possible concurrent instances across all ESs

$$\min_{\substack{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{a}\\ \boldsymbol{\beta},\boldsymbol{p},\boldsymbol{x}}} \quad \sum_{m\in\mathcal{M}} \tau_m^{\text{e2e}} \tag{15a}$$

$$\text{s.t.} \quad (14c), (14d), (14e), \tag{15b}$$

$$\sum_{n\in\mathcal{N}} \sum_{t\in\mathcal{T}} x_{nt} \le NT. \tag{15c}$$

Functions 1 to 5 are tailored to exploit the structure of OPLA and are discussed next. For brevity, we define the variable set as $\boldsymbol{\Psi} \triangleq \{\boldsymbol{\lambda}, \boldsymbol{f}, \boldsymbol{a}, \boldsymbol{\beta}, \boldsymbol{p}, \boldsymbol{x}\}$. To approximate the optimal solution of the reformulated problem (15), we propose an exploitation–exploration procedure (EEP) based on the *explore-then-commit* principle [46], as detailed in Algorithm 1. This principle guides both the algorithm design and the tuning of the penalty parameter $\rho \in [0, \rho_{\max}]$, which enforces the binary constraint $a_{\text{mn}}(1 - a_{\text{mn}}) = 0$. Larger values of ρ more strongly penalize infeasible (nonbinary) solutions [37], [47], but may cause Function 2 to converge to overly conservative solutions, in which all $a_{\text{mn}} = 0$, corresponding to full local processing and higher latency. Conversely, smaller ρ values allow more flexibility in the admission decision that might yield lower latency solutions at the cost of feasibility. EEP balances this tradeoff by gradually increasing ρ, shifting from exploration to exploitation.

The interval $[0, \rho_{\max}]$ is discretized into $\lfloor \rho_{\max}/\rho_{\texttt{step}} \rfloor + 1$ grid points, treating each ρ as an *arm* in a finite multiarmed bandit setting. Unlike classical stochastic bandits [46], our setting is deterministic, so each arm is evaluated once, ensuring zero post-tuning regret and linear exploration cost. In the *exploration* phase (Lines 2–4, Algorithm 1), the algorithm evaluates solution quality for each candidate ρ, a process well-suited for parallelization. Once the best ρ is identified, the algorithm enters the exploitation phase (Lines 5–7, Algorithm 1), where the admission decisions a_{mn} are rounded and fixed for the rest of the optimization.

A. Presolve Heuristic for x_{nt} (Function 1)

To simplify the highly coupled joint user admission and concurrency scheduling problem, we predetermine a subset of the scheduling variables x_{nt}. This reformulation reduces the problem to a binary decision task: either accept ($a_{\text{mn}} = 1$) or reject ($a_{\text{mn}} = 0$) a proposed schedule, while preserving feasibility. Given the importance of latency minimization in edge environments, our strategy sets all x_{nt} entries corresponding to the first "1" in each row of the matrix $\boldsymbol{B}$ in constraint (7) to "1," as implemented in Function 1. This ensures that each request is scheduled for execution at the earliest available slot.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

To illustrate, we consider the equivalent form of constraint (7), as shown in (A.1). Let $(M, N) = (2, 2)$, $\boldsymbol{B} = \left[\begin{smallmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{smallmatrix}\right]$ and assume three time slots per ES. Constraint (7) becomes

$$\begin{bmatrix} 1&1&0&0&0&0 \\ 0&1&1&0&0&0 \\ 0&0&0&1&0&0 \\ 0&0&0&0&1&1 \end{bmatrix} \begin{bmatrix} x_{11} \\ x_{12} \\ x_{13} \\ x_{21} \\ x_{22} \\ x_{23} \end{bmatrix} \geqslant \begin{bmatrix} a_{11} \\ a_{21} \\ a_{12} \\ a_{22} \end{bmatrix}. \tag{16}$$

By applying the presolve scheme to this setup, we derive a proposed schedule for (16), given by $(x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23}) = (1, 1, 0, 1, 1, 0)$.

This heuristic effectively linearizes the nonconvex quadratic disjunctive constraint (13) by first relaxing constraint (14e). Consequently, (13) can be reformulated as

$$\text{For } t=1: \ \frac{S_m C_m T x_{nt} a_{\mathrm{mn}}}{J} \leq f_{\mathrm{mn}}^{\mathtt{edg}}, \ \forall m, n \tag{17a}$$

$$\text{For } t\geq 2: \bigvee_{t\geq 2}\left[\frac{S_m C_m T x_{nt} a_{\mathrm{mn}}}{Jt} \leq f_{\mathrm{mn}}^{\mathtt{edg}} \leq \frac{S_m C_m T x_{nt} a_{\mathrm{mn}}}{J(t-1)}\right], \quad \forall m \in \mathcal{M}, n \in \mathcal{N}, t \in \mathcal{T}. \tag{17b}$$

By imposing an upper bound on (17a) using constraint (11b), we obtain

$$\text{For } t=1: \frac{S_m C_m T x_{nt} a_{\mathrm{mn}}}{J} \leq f_{\mathrm{mn}}^{\mathtt{edg}} \leq a_{\mathrm{mn}} F_n^{\max}. \tag{18}$$

Assuming that the edge processing time for each connected user fits within a single time slot, the presolve procedure in Function 1 facilitates the linearization of the boolean expressions in (17b) and (18) into the following linear system, $\forall m \in \mathcal{M}$, $\forall n \in \mathcal{N}$, $\forall t \in \mathcal{T}$:

$$\frac{S_m C_m T a_{\mathrm{mn}} x_{nt}}{J} + \sum_{t\geq 2} \frac{S_m C_m T a_{\mathrm{mn}} x_{nt}}{Jt} \leq f_{\mathrm{mn}}^{\mathtt{edg}}, \tag{19a}$$

$$f_{\mathrm{mn}}^{\mathtt{edg}} \leq a_{\mathrm{mn}} F_n^{\max} + \sum_{t\geq 2} \frac{S_m C_m T a_{\mathrm{mn}} x_{nt}}{J(t-1)}, \tag{19b}$$

$$\sum_{t\in\mathcal{T}} a_{\mathrm{mn}} x_{nt} \leq 1. \tag{19c}$$

B. First Stage SCA (Function 2)

After determining x_{nt} using Function 1, the next step involves optimizing the variables $(\boldsymbol{\lambda}, \boldsymbol{f}, \boldsymbol{a}, \boldsymbol{\beta}, \boldsymbol{p})$ as defined in (15). To address the binary nature of $\boldsymbol{a}$, we relax it to a continuous variable, allowing $a_{\mathrm{mn}} \in [0, 1]$. However, directly solving this relaxed problem often yields suboptimal solutions. To enforce the binary constraint $a_{\mathrm{mn}} \in \{0, 1\}$, we incorporate a penalty function $\sum_{m\in\mathcal{M}} \sum_{n\in\mathcal{N}} (a_{\mathrm{mn}} - a_{\mathrm{mn}}^2) \geq 0$, $\forall a_{\mathrm{mn}} \in \{0, 1\}$ into the objective, where the term $a_{\mathrm{mn}} - a_{\mathrm{mn}}^2$ is minimized at $a_{\mathrm{mn}} = 1$ or $a_{\mathrm{mn}} = 0$, effectively encouraging binary outcomes. The resulting regularized relaxed optimization problem is formulated as

$$\min_{\substack{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{a} \\ \boldsymbol{\beta},\boldsymbol{p}}} \ \sum_{m\in\mathcal{M}} \tau_m^{\mathtt{e2e}} + \rho \sum_{m\in\mathcal{M}} \sum_{n\in\mathcal{N}} (a_{\mathrm{mn}} - a_{\mathrm{mn}}^2) \tag{20a}$$

$$\text{s.t.} \quad (14c), (14d), \tag{20b}$$

$$a_{\mathrm{mn}} \in [0, 1], \ \forall m \in \mathcal{M}, n \in \mathcal{N} \tag{20c}$$

where ρ is the penalty parameter to balance the first and second terms of the objective function in (20a). The choice of ρ is discussed in Algorithm 1.

To address the nonconvexity of the objective (20a) and constraint (14d), we employ the SCA framework [36] to derive their convex upper bounds. To do so, let $x^{(i)}$ be a feasible point of x found at the ith iteration of an iterative algorithm presented shortly. The convex upper bounds of the bilinear function $f_{\mathtt{bil}}(x_1, x_2) \triangleq x_1 x_2$ and the rational function $f_{\mathtt{rat}}(x_1, x_2, x_3) \triangleq \frac{x_1 x_2}{x_3}$ around the feasible points $(x_1^{(i)}, x_2^{(i)})$ can be found as

$$f_{\mathtt{bil}}(x_1, x_2) \leq F_{\mathtt{bil}}^{(i)}(x_1, x_2) \triangleq \frac{x_2^{(i)}}{2x_1^{(i)}} x_1^2 + \frac{x_1^{(i)}}{2x_2^{(i)}} x_2^2 \tag{21a}$$

$$f_{\mathtt{rat}}(x_1, x_2, x_3) \leq F_{\mathtt{rat}}^{(i)}(x_1, x_2, x_3) \triangleq \frac{x_2^{(i)}}{2x_1^{(i)}} \frac{x_1^2}{x_3} + \frac{x_1^{(i)}}{2x_2^{(i)}} \frac{x_2^2}{x_3} \tag{21b}$$

with $(x_1, x_2, x_3) \in \mathbb{R}_{++}$. We note that $f_{\mathtt{bil}}(x_1, x_2) = F_{\mathtt{bil}}^{(i)}(x_1, x_2)$ and $f_{\mathtt{rat}}(x_1, x_2, x_3) = F_{\mathtt{rat}}^{(i)}(x_1, x_2, x_3)$ once $(x_1^{(i)}, x_2^{(i)}) := (x_1^{(i-1)}, x_2^{(i-1)})$. The right-hand sides of (21a) and (21b) can be reformulated as convex constraints. In particular, the inequality $x^2 \leq t$ can be equivalently expressed as the second-order cone (SOC) constraint $\|x; \frac{t-1}{2}\|_2 \leq \frac{t+1}{2}$. Therefore, the constraints $x_1^2 \leq t$ and $x_2^2 \leq t$ admit SOC representations. Similarly, the inequality $x^2/y \leq t$ can be equivalently reformulated as $\|x; \frac{1}{2}(t-y)\|_2 \leq \frac{1}{2}(t+y)$.

By applying inequality (21a) at the feasible point $(f_m^{\mathtt{loc},(i)}, \lambda_{\mathrm{mn}}^{(i)})$ of $\tau_m^{\mathtt{loc}} = \frac{(1-\sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}}) S_m C_m}{f_m^{\mathtt{loc}}}$, we obtain the following convex upper bound $\tau_m^{\mathtt{loc},(i)}$:

$$\tau_m^{\mathtt{loc}} \leq \tau_m^{\mathtt{loc},(i)} \triangleq \frac{S_m C_m \left(1 - \sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}}\right)^2}{2 f_m^{\mathtt{loc},(i)} \left(1 - \sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}}^{(i)}\right)} + \frac{S_m C_m f_m^{\mathtt{loc},(i)} \left(1 - \sum_{n\in\mathcal{N}} \lambda_{\mathrm{mn}}^{(i)}\right)}{2(f_m^{\mathtt{loc}})^2}. \tag{22}$$

We note that the last term on the right-hand side of (22) admits the SOC representation. To show this, we first introduce an auxiliary variable $z \geq 0$. The inequality $\frac{1}{x^2} \leq t$ can be equivalently written $\frac{1}{x^2} \leq z^2 \leq t$, which implies the pair of constraints: $z^2 \leq t \wedge 1 \leq zx$. These can be reformulated as two SOC constraints: $\|z; \frac{t-1}{2}\|_2 \leq \frac{t+1}{2} \wedge \|1; \frac{z-x}{2}\|_2 \leq \frac{z+x}{2}$.

Similarly, applying inequality (21b) around the feasible point $(a_{\mathrm{mn}}^{(i)}, \lambda_{\mathrm{mn}}^{(i)})$ for $\tau_{\mathrm{mn}}^{\mathtt{edg}} = S_m C_m \frac{a_{\mathrm{mn}} \lambda_{\mathrm{mn}}}{f_{\mathrm{mn}}^{\mathtt{edg}}}$ yields

$$\tau_{\mathrm{mn}}^{\mathtt{edg}} \leq \tau_{\mathrm{mn}}^{\mathtt{edg},(i)} \triangleq S_m C_m \left(\frac{\lambda_{\mathrm{mn}}^{(i)}}{2a_{\mathrm{mn}}^{(i)}} \frac{a_{\mathrm{mn}}^2}{f_{\mathrm{mn}}^{\mathtt{edg}}} + \frac{a_{\mathrm{mn}}^{(i)}}{2\lambda_{\mathrm{mn}}^{(i)}} \frac{\lambda_{\mathrm{mn}}^2}{f_{\mathrm{mn}}^{\mathtt{edg}}}\right) \tag{23}$$

where $\tau_{\mathrm{mn}}^{\mathtt{edg},(i)}$ is a convex upper bound of $\tau_{\mathrm{mn}}^{\mathtt{edg}}$.

To derive the convex upper bound for $\tau_{\mathrm{mn}}^{\mathtt{wls}}$, we introduce a set of auxiliary decision variables $\boldsymbol{\nu} \triangleq \{\nu_{\mathrm{mn}}\}_{\forall m,n}$, which act as affine lower bounds for $r_{\mathrm{mn}}(\beta_{\mathrm{mn}}, p_{\mathrm{mn}})$. In particular, we impose the constraint

$$r_{\mathrm{mn}}(\beta_{\mathrm{mn}}, p_{\mathrm{mn}}) \geq \nu_{\mathrm{mn}}, \ \forall m \in \mathcal{M}, n \in \mathcal{N} \tag{24}$$

which is convex. To verify its convexity, we consider the inequality: $y \log_2\left(1 + \frac{x}{y}\right) \geq t$. Since the Hessian of $y \log_2\left(1 + \frac{x}{y}\right)$ is negative semidefinite over the domain

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

$(x \geq 0, y > 0)$, we conclude that the function is jointly concave in (x, y). By substituting $x = p_{\mathrm{mn}}\|\boldsymbol{h}_{\mathrm{mn}}\|^2/N_0$, $y = \beta_{\mathrm{mn}}W$, and $t = v_{\mathrm{mn}}$, it follows directly that (24) represents a convex constraint. As a result, we have $\tau^{\mathtt{wls}}_{\mathrm{mn}} \leq a_{\mathrm{mn}}\lambda_{\mathrm{mn}}S_m/v_{\mathrm{mn}}$. Applying (21b) around the feasible point $(a^{(i)}_{\mathrm{mn}}, \lambda^{(i)}_{\mathrm{mn}})$, we obtain the following convex upper bound $\tau^{\mathtt{wls},(i)}_{\mathrm{mn}}$:

$$\tau^{\mathtt{wls}}_{\mathrm{mn}} \leq \tau^{\mathtt{wls},(i)}_{\mathrm{mn}} \triangleq S_m\left(\frac{\lambda^{(i)}_{\mathrm{mn}}}{2a^{(i)}_{\mathrm{mn}}}\frac{a^2_{\mathrm{mn}}}{v_{\mathrm{mn}}}+\frac{a^{(i)}_{\mathrm{mn}}}{2\lambda^{(i)}_{\mathrm{mn}}}\frac{\lambda^2_{\mathrm{mn}}}{v_{\mathrm{mn}}}\right). \tag{25}$$

From the above approximations, we iteratively replace $\tau^{\mathtt{e2e}}_m$ by the approximate convex function $\tau^{\mathtt{e2e},(i)}_m = \tau^{\mathtt{loc},(i)}_m + \sum_{n\in\mathcal{N}}\left(\tau^{\mathtt{edg},(i)}_{\mathrm{mn}} + \tau^{\mathtt{wls},(i)}_{\mathrm{mn}}\right)$. In addition, constraint (14d) is convexified by $\tau^{\mathtt{e2e},(i)}_m \leq \tau^{\mathtt{max}}_m$, $\forall m \in \mathcal{M}$.

We are now in a position to handle the nonconvexity of the penalty function $\rho\sum_{m\in\mathcal{M}}\sum_{n\in\mathcal{N}}(a_{\mathrm{mn}} - a^2_{\mathrm{mn}})$. By $\omega \triangleq \sum_{m\in\mathcal{M}}\sum_{n\in\mathcal{N}}(a_{\mathrm{mn}} - a^2_{\mathrm{mn}})$, we approximate it around the feasible point $a^{(i)}_{\mathrm{mn}}$ as

$$0 \leq \omega \leq \omega^{(i)} \triangleq \sum_{m\in\mathcal{M}}\sum_{n\in\mathcal{N}}\left(\left(a^{(i)}_{\mathrm{mn}}\right)^2 - 2\, a_{\mathrm{mn}}a^{(i)}_{\mathrm{mn}} + a_{\mathrm{mn}}\right). \tag{26}$$

Summing up, we solve the following convex approximate program of (20) at iteration i:

$$\min_{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{a},\boldsymbol{\beta},\boldsymbol{p},\boldsymbol{v}} L(\boldsymbol{\Psi}^{(i)}_1) \triangleq \sum_{m\in\mathcal{M}}\tau^{\mathtt{e2e},(i)}_m + \rho\omega^{(i)} \tag{27a}$$

$$\text{s.t.} \quad (6),\ (8),\ (9),\ (10),\ (11),\ (12),\ (19),\ (24) \tag{27b}$$

$$\tau^{\mathtt{e2e},(i)}_m \leq \tau^{\mathtt{max}}_m,\ \forall m \in \mathcal{M} \tag{27c}$$

$$a_{\mathrm{mn}} \in [0,1],\ \forall m \in \mathcal{M},\ n \in \mathcal{N} \tag{27d}$$

where $\boldsymbol{\Psi}_1 \triangleq (\boldsymbol{\lambda}, \boldsymbol{f}, \boldsymbol{a}, \boldsymbol{\beta}, \boldsymbol{p})$. We iteratively solve (27) until convergence or a stopping condition is met, with the overall algorithm summarized in Function 2. To initialize a feasible point, we set $\lambda^{(0)}_{\mathrm{mn}} = a^{(0)}_{\mathrm{mn}} = 1/N$ and $f^{\mathtt{loc},(0)}_m = F^{\mathtt{max}}_{\mathtt{loc}}$.

C. Rounding Procedure (Function 3)

We observe numerically that Function 2 may not always offer a complete binary solution. The next step involves rounding the matrix $\boldsymbol{A}_{M\times N}$ to obtain $\boldsymbol{A}^* \leftarrow \lfloor \boldsymbol{A}_{M\times N} + \frac{1}{2}\rfloor$. If the resulting matrix $\boldsymbol{A}^*$ is binary (i.e., all entries are either 0 or 1), the result is accepted (line 4). However, if $\boldsymbol{A}^*$ consists entirely of zeros, each row of $\boldsymbol{A}_{M\times N}$ must be revisited. In particular, for rows where elements lie within the range $1/N \pm \mathtt{tolerance}$, only the maximum element is rounded to 1 (lines 14 and 15), while all other entries in the row are set to 0 to satisfy constraint (6). In this procedure, we set the tolerance to 1%. The complete rounding mechanism is detailed in Function 3.

D. Second Stage SCA (Function 3)

It is evident that the rounded matrix $\boldsymbol{A}_{\mathtt{rounded}}$ may lead to an infeasible solution for the OPLA problem in (15). To mitigate this issue and refine the optimal continuous solution, we solve the following subproblem, considering that the decision variables $(\boldsymbol{x}^\star, \boldsymbol{a}^\star)$ are fixed:

$$\min_{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{\beta},\boldsymbol{p}} \sum_{m\in\mathcal{M}}\hat{\tau}^{\mathtt{e2e}}_m \tag{28a}$$

$$\text{s.t.} \quad (8),\ (9),\ (10),\ (11),\ (12),\ (14d) \tag{28b}$$

where $\hat{\tau}^{\mathtt{e2e}}_m$ denotes the e2e latency computed using the fixed values $(\boldsymbol{x}^\star, \boldsymbol{a}^\star)$. This subproblem allows us to fine-tune the continuous variables while ensuring feasibility with respect to the original problem constraints.

Following Section IV-B, we only need to reapproximate $\tau^{\mathtt{edg}}_{\mathrm{mn}}$ and $\tau^{\mathtt{wls}}_{\mathrm{mn}}$ as they are independent of $(\boldsymbol{x}^\star, \boldsymbol{a}^\star)$. Let be $\hat{\tau}^{\mathtt{edg}}_{\mathrm{mn}} = \frac{a^\star_{\mathrm{mn}}\lambda_{\mathrm{mn}}S_mC_m}{f^{\mathtt{edg}}_{\mathrm{mn}}}$ and $\hat{\tau}^{\mathtt{wls}}_{\mathrm{mn}} = \frac{a^\star_{\mathrm{mn}}\lambda_{\mathrm{mn}}S_m}{r_{\mathrm{mn}}(\beta_{\mathrm{mn}},p_{\mathrm{mn}})}$. Their convex upper bounds, obtained using the SCA method, are given by:

$$\hat{\tau}^{\mathtt{edg},(i)}_{\mathrm{mn}} = S_mC_m\left(\frac{\lambda^2_{\mathrm{mn}}}{2\lambda^{(i)}_{\mathrm{mn}}f^{\mathtt{edg},(i)}_{\mathrm{mn}}} + \frac{\lambda^{(i)}_{\mathrm{mn}}f^{\mathtt{edg},(i)}_{\mathrm{mn}}}{2\left(f^{\mathtt{edg}}_{\mathrm{mn}}\right)^2}\right) \tag{29a}$$

$$\hat{\tau}^{\mathtt{wls},(i)}_{\mathrm{mn}} = S_m\left(\frac{\lambda^2_{\mathrm{mn}}}{2\lambda^{(i)}_{\mathrm{mn}}v^{(i)}_{\mathrm{mn}}}+\frac{\lambda^{(i)}_{\mathrm{mn}}v^{(i)}_{\mathrm{mn}}}{2v^2_{\mathrm{mn}}}\right). \tag{29b}$$

It is noted that constraints (29) hold true for $a^\star_{\mathrm{mn}} = 1$; otherwise, we set $\hat{\tau}^{\mathtt{edg}}_{\mathrm{mn}} = \hat{\tau}^{\mathtt{wls}}_{\mathrm{mn}} = 0$. As a result, we replace the objective (28a) by the approximate convex function $\hat{\tau}^{\mathtt{e2e},(i)}_m = \tau^{\mathtt{loc},(i)}_m + \sum_{n\in\mathcal{N}}\left(\hat{\tau}^{\mathtt{edg},(i)}_{\mathrm{mn}} + \hat{\tau}^{\mathtt{wls},(i)}_{\mathrm{mn}}\right)$.

At iteration i, we solve the following convex approximation of the subproblem (28):

$$\min_{\substack{\boldsymbol{\lambda},\boldsymbol{f},\boldsymbol{\beta}\\ \boldsymbol{p},\boldsymbol{v}}} L(\boldsymbol{\Psi}^{(i)}_2) \triangleq \sum_{m\in\mathcal{M}}\hat{\tau}^{\mathtt{e2e},(i)}_m \tag{30a}$$

$$\text{s.t.} \quad (8),\ (9),\ (10),\ (11),\ (12),\ (24) \tag{30b}$$

$$\hat{\tau}^{\mathtt{e2e},(i)}_m \leq \tau^{\mathtt{max}}_m,\ \forall m \in \mathcal{M} \tag{30c}$$

where $\boldsymbol{\Psi}_2 \triangleq (\boldsymbol{\lambda}, \boldsymbol{f}, \boldsymbol{\beta}, \boldsymbol{p})$ denotes the set of optimization variables. The procedure is detailed in Function 4.

E. Postsolve Heuristic for x_{Nt} (Function 3)

When an allocated slot on an ES is rejected (e.g., $a_{\mathrm{mn}} = 0$, we update all corresponding presolve variables by setting $x_{nt} = 0$ to maintain logical consistency. For example, suppose $a_{11} = a_{21} = 1$ and $a_{12} = a_{22} = 0$; then the presolve assignment $(x_{11}, x_{12}, x_{13}, x_{21}, x_{22}, x_{23})$ from Section IV-A updated to $(1, 1, 0, 0, 0, 0)$. The steps for this update are summarized in Function 5.

F. Complexity Analysis

We now analyze the complexity of Functions 1–5 and eventually the overall complexity of the EEP approach.

First, the complexity of the optimization problem addressed by Function 2 is analyzed in terms of its constraints, variables, and iterations. In particular, this algorithm has approximately $x_1 = 1+3M+N+7MN+2MNT$ constraints and $y_1 = 6MN+M$ scalar variables per iteration. Then, let Z_1 and Z_2 be the number of iterations for Functions 2 and 4, respectively. From this, the estimated ε-accuracy complexity of Function 2 is dominated by $\approx \mathcal{O}\big(Z_1\log(1/\varepsilon)\big[\big(2(y_1+1)+x_1\big)^3+x_1\big]\big)$ [48]. Similarly, Function 4 operates with approximately $x_2 = 1+2M+N+6MN$ constraints and $y_2 = 5MN+M$ decision variables, yielding an estimated complexity that is dominated by $\approx \mathcal{O}\big(Z_2\log(1/\varepsilon)\big[\big(2(y_2+1)+x_2\big)^3+x_2\big]\big)$.

On the other hand, Functions 1, 3, and 5 primarily perform linear search and variable assignments. Their complexities are dominated by the sizes of the arrays they operate on [49], resulting in $\mathcal{O}(MN^2T)$, $\mathcal{O}(MN)$, and $\mathcal{O}(NT)$, respectively.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYSTEM PARAMETERS

Parameters	Value
System bandwidth (W)	10 MHz
Noise power spectral density (N_0)	-174 dBm/Hz
Number of UEs (M)	10
Number of ESs (N)	5
Processing capacity of UEs ($F_{\text{loc}}^{\max}$)	1 GHz
Processing capacity of ESs ($F_n^{\max}$)	4 GHz
Task size (S_m)	1354 bytes
Task complexity (C_m)	400 cycles/byte
UE's maximum transmitted power ($P_m^{\max}$)	23 dBm
Number of antennas at co-located AP (K)	2
Maximum latency ($\tau_{\max}$)	18 ms
Step size (ρ_{step})	1
Maximum penalty ($\rho_{\max}$)	5
Time slot per ES (T)	3
Single time slot duration (J/T)	8.3 ms

Having this, the overall complexity of the EEP procedure is approximately $\approx \mathcal{O}\left((\lfloor\rho_{\max}/\rho_{\text{step}}\rfloor+1)Z_1\log(1/\varepsilon)\left[(2(y_1+1)+x_1)^3+x_1\right]+Z_2\log(1/\varepsilon)\left[(2(y_2+1)+x_2)^3+x_2\right]+MN^2T+MN+NT\right)$.

V. PERFORMANCE EVALUATION

A. Simulation Setup and System Parameters

1) Network: We consider a small-cell environment where UEs and ESs are randomly deployed within a 100×100 m^2 rectangular area. The large-scale fading φ_{mn} of the uplink channel between UE m and ES n is modeled as $\varphi_{\text{mn}} = 10^{\text{PL}(d_{\text{mn}})/10}$, where d_{mn} is the distance between UE m and ES n, and the path loss function is defined as $\text{PL}(d_{\text{mn}}) = -35.3 - 37.6\log_{10}(d_{\text{mn}})$ (dB), following the model in [50].

Table II summarizes the key simulation parameters used in our evaluation. Task sizes S_m and computational complexities C_m are drawn from realistic edge workloads as characterized in [30]. Transmit power limits $P_m^{\max}$, latency tolerance thresholds $\tau_{\max}$, time slot durations J/T, and maximum processing capacity $F_n^{\max}$ follow values adopted in prior studies [50], [51], [52], [53].

2) Performance Metrics: To assess the system performance, we consider the following metrics.

1) Average latency measures the mean sum latency experienced by UEs.
2) Average concurrency instance usage reflects the average number of active time slots used for offloading across all UEs.
3) Average Jain's index $\mathcal{J}$ for bandwidth allocation

$$\mathcal{J} = \frac{1}{|\mathcal{I}|}\left(\sum_{(m,n)\in\mathcal{I}}\beta_{\text{mn}}\right)^2\left(\sum_{(m,n)\in\mathcal{I}}\beta_{\text{mn}}^2\right)^{-1} \tag{31}$$

where $\mathcal{I} := \{(m,n)|\beta_{\text{mn}} > 0\}$ denotes the set of UE–ES pairs with nonzero bandwidth. A value of $\mathcal{J}$ close to 1 indicates equitable bandwidth allocation [54].

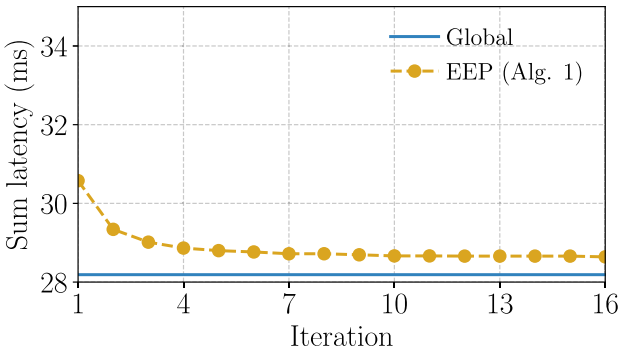


Fig. 3. Convergence behavior of EEP (see Algorithm 1).

4) Average edge resource utilization: the utilization ratio δ is calculated as

$$\delta = \frac{\sum_m\sum_n f_{\text{mn}}^{\text{edg}}}{NF_n^{\max}}$$

where the numerator aggregates the allocated edge computing resources, and the denominator represents the total available capacity across all ESs. Higher values (closer to 1) indicate efficient use of edge resources, whereas lower values suggest underutilization.

Benchmark schemes: To evaluate the performance of Algorithm 1, referred to as "EEP (see Algorithm 1)," we compare it against the following baseline approaches.

1) "*Nearest Server*": Each UE is assigned to its geographically closest ES [55].
2) "*Random Server*": Each UE is assigned to a randomly selected ES [20].
3) "*Random Offload*": The offloading ratio λ_{mn} is randomly selected for each UE–ES pair, following the method in [21].
4) "*Global*": A global optimal solution is computed using the branch-and-reduce method implemented in BARON [56].

B. Numerical Results and Discussion

1) Convergence Behavior: Fig. 3 illustrates the convergence behavior of the proposed approach under a task complexity of $C_m = 400$ cycles/byte. The figure combines the iterative processes of Functions 2 and 4, showing that the algorithm converges within about 16 iterations. The sum latency begins at 30.57 ms and gradually decreases, stabilizing around 28.7 ms. The EEP method achieves a final sum latency of 28.64 ms, which is only slightly higher than the global optimal value of 28.18 ms, thereby confirming its effectiveness.

To justify the choice of $\rho_{\max} = 5$, we conduct additional numerical sensitivity experiments with values up to $\rho = 1000$. We have numerically observed that moderate values of ρ (around 4–5) offer the best balance between convergence speed and solution quality. In contrast, very large values of ρ lead to many more iterations and significantly worse optimality gaps. This occurs because, when ρ is excessively large, the penalty

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

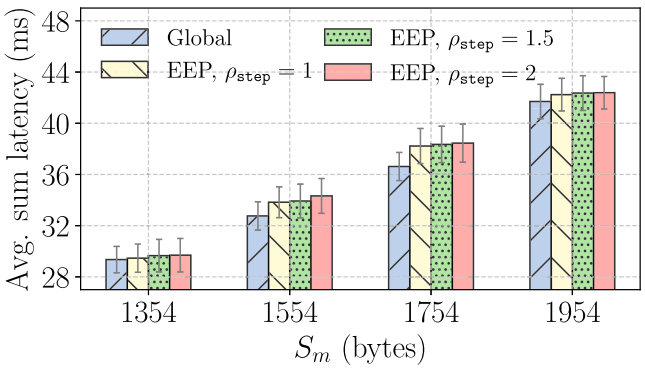


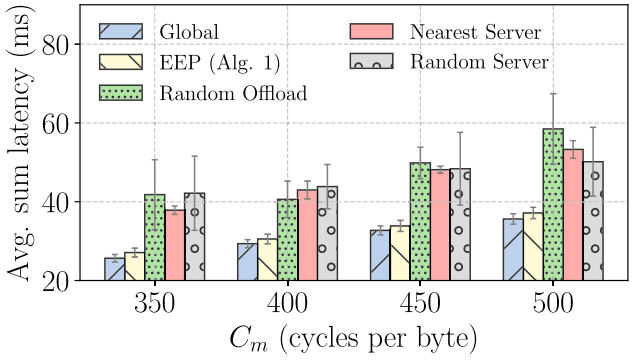
Fig. 4. Latency performance of EEP with varying ρ_{step}.

term $\rho\omega^{(i)}$ in (27) dominates the objective, causing the solver to prioritize binary over latency minimization and resulting in overly conservative, suboptimal admission decisions.

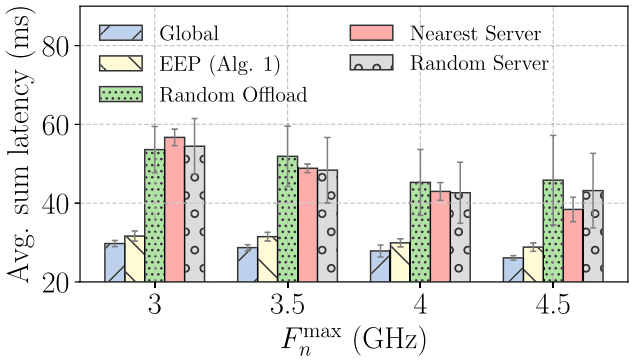
2) Effect of ρ_{step}: Fig. 4 illustrates how the step size ρ_{step} influences performance under different task sizes (S_m = 1354–1954 bytes). For instance, when S_m = 1354, the global method attains an average latency of 29.35 ms, whereas the proposed EEP achieves 29.46, 29.65, and 29.69 ms for ρ_{step} = 1, 1.5, and 2, respectively. This behavior is consistently observed across all task sizes. Overall, the results indicate that larger step sizes lead to higher average latency, suggesting that a finer exploration of ρ yields better solution quality.

3) Latency Performance: The evolution of the average sum latency with various values of task complexity C_m, ESs' processing capacity $F_n^{\max}$, and task size S_m is shown in Fig. 5(a)–(c), respectively. As expected, the global method consistently achieves the lowest latency, serving as a performance benchmark. The proposed EEP (see Algorithm 1) method closely tracks this benchmark, delivering near-optimal results across all scenarios. As C_m increases, global maintains latencies in the range of 29.65–35.62 ms [see Fig. 5(a)]. EEP remains within 1.14–1.53 ms of this range, demonstrating its effectiveness. In contrast, baseline methods, such as nearest server, random offload, and random server, incur higher latencies, with random server exceeding 48 ms. When $F_n^{\max}$ increases, global method achieve a latency of around 29–26 ms, with EEP trailing by just 1.9–2.74 ms, as shown in Fig. 5(b). Meanwhile, the other approaches exhibit a latency of 38–56 ms, underscoring the advantage of EEP's design. Finally, as S_m grows, latency naturally increases for all methods [see Fig. 5(c)]. Nonetheless, EEP remains close to global, with only a 0.4–0.5 ms difference. In comparison, nearest server and random offload surpass 103 ms for task sizes above 1504 bytes, and random server consistently performs the worst. Overall, EEP achieves near-optimal performance with minimal overhead, validating its robustness and practicality. In contrast, randomized baselines lack consistency and degrade significantly under higher task complexity and load.

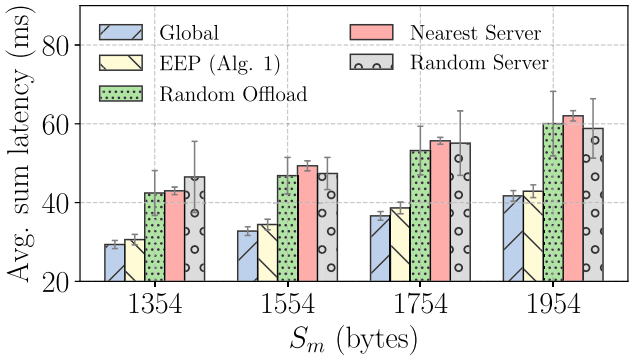
4) Utilization of Time Slots: Fig. 6(a)–(c) illustrates the average concurrency instance usage under varying the task complexity, edge processing capacity, and task size, respectively. Across all scenarios, the nearest server method consistently incurs the highest resource usage, averaging



(a) Latency versus task complexity.



(b) Latency versus processing capacity of ESs.

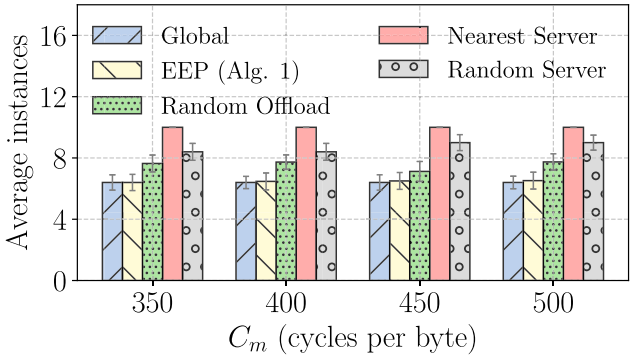


(c) Latency versus task size.

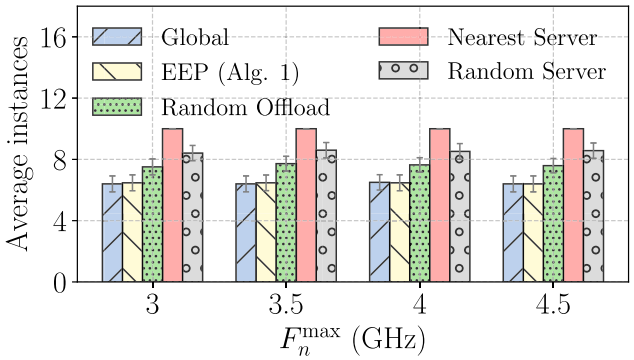
Fig. 5. Average sum latency performance of the compared algorithms with varying (a) C_m, (b) $F_n^{\max}$, and (c) S_m.

around 10 active instances. In contrast, the global and EEP methods demonstrate the most efficient resource utilization, requiring only about 6.4–6.52 instances regardless of parameter changes. Random server and random offload exhibit moderate efficiency, with average usage of approximately 8.3–9 and 7–8 instances, respectively. This performance hierarchy remains consistent across all test conditions shown in Fig. 6(a)–(c). The key finding is the stability and efficiency of the global and EEP approaches, which maintain minimal concurrency requirements across all workloads. Nearest server demands roughly one-third more resources compared to these methods, while random-based approaches deliver only marginal gains in comparison.

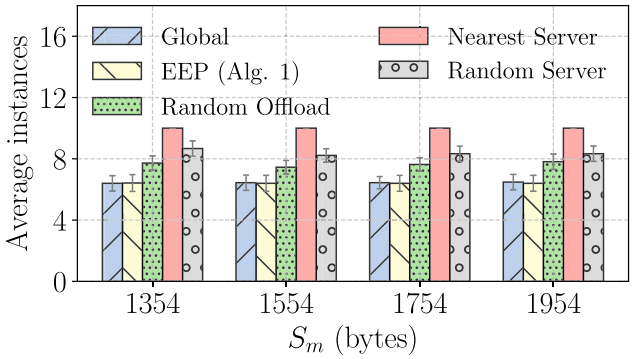
Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.



(a) Concurrency instance versus task complexity.

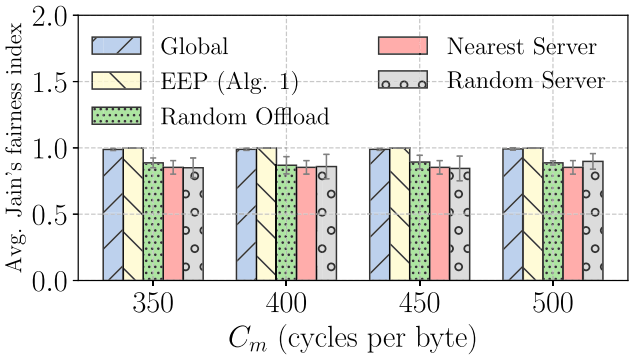


(b) Concurrency instance versus processing capacity of ESs.

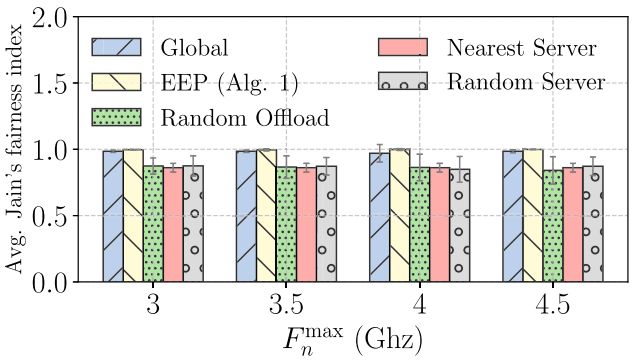


(c) Concurrency instance versus task size.

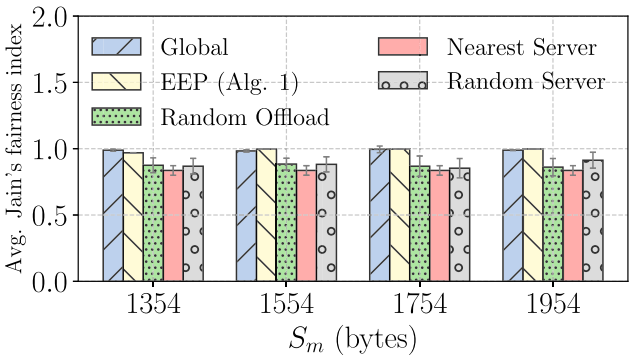
Fig. 6. Average concurrency invocation of the compared algorithms with varying (a) C_m, (b) $F_n^{\max}$, and (c) S_m.



(a) Bandwidth fairness versus task complexity.



(b) Bandwidth fairness versus processing capacity of ESs



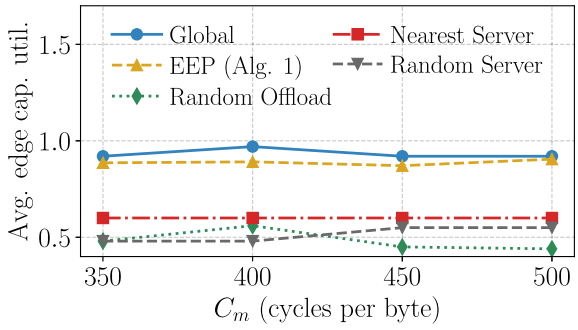
(c) Bandwidth fairness versus task size.

Fig. 7. Average Jain's fairness index for β_{mn} of the compared algorithms with varying. (a) C_m, (b) $F_n^{\max}$, and (c) S_m.

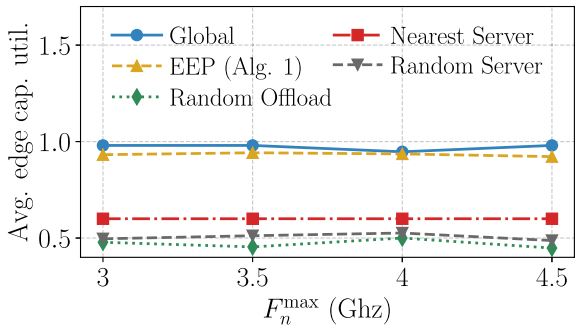
5) Bandwidth Allocation: Across all system settings, including task complexity in Fig. 7(a), ES processing capacity in Fig. 7(b), and task size in Fig. 7(c), EEP and global method consistently delivers the highest bandwidth fairness, with Jain's index values approaching 1 (approximately 0.98–0.99). Interestingly, EEP maintains this high level of fairness regardless of variations in workload or computational resources. In contrast, nearest server, random server, and random offload perform moderately, with fairness scores ranging from 0.84 to 0.87. Overall, the most prominent result is EEP's robustness in achieving equitable bandwidth distribution across all scenarios, independent of system complexity or scale.

6) Edge Capacity Utilization: Fig. 8(a)–(c) depicts server utilization across varying task complexity, ES processing capacity, and task size, respectively. As task complexity increases, both the global and EEP methods maintain high utilization levels, with global around 0.92–0.98, and EEP around 0.87–0.94, respectively. In contrast, the nearest server approach achieves significantly lower utilization (around 0.6), while random server and random offload methods range between 0.44 and 0.57. This disparity highlights the inefficiency of simplistic or random allocation strategies in effectively utilizing edge resources. When edge processing capacity varies [see Fig. 8(b)], global and EEP consistently sustain high utilization (0.92–0.98), whereas the nearest server and random methods exhibit notably lower levels (0.45–0.60). A similar observation is seen with increasing task size [see Fig. 8(c)], where global and EEP remain near full utilization

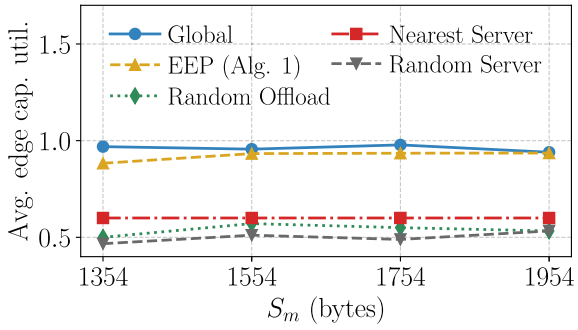
Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.



(a) Utilization versus task complexity.



(b) Utilization versus processing capacity of ESs.



(c) Utilization versus task size.

Fig. 8. Average edge capacity utilization of the compared algorithms with varying. (a) C_m, (b) $F_n^{\max}$, and (c) S_m.

(0.88–0.98), while baseline methods again underperform, such as nearest server around 0.6 and random schemes between 0.47 and 0.57. Overall, these results highlight that effective resource utilization is closely associated with lower latency performance, as previously shown in Fig. 5, affirming the efficiency of the proposed EEP method.

7) Impact of Heterogeneous UE–ES Configurations: Fig. 9 presents the average sum latency for heterogeneous configurations of M users and N servers. Across these scenarios, the relative difference between the EEP method and the global method remains below 6%. For the network configuration of size $(M, N) = (10, 5)$, the latency of EEP is 30.36 ms, which is 5.93% higher than the global method (28.66 ms). For the $(M, N) = (20, 10)$ configuration, the latency difference is about 1.3 ms. In the larger network size of $(M, N) = (40, 20)$, the global method achieves a latency of 149.48 ms, while the EEP method is 5.90 ms higher than the global benchmark. For

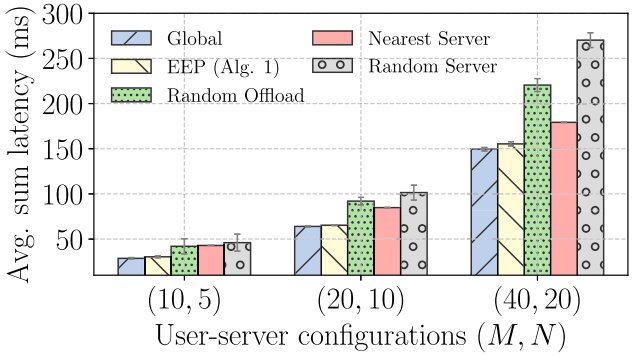


Fig. 9. Average sum latency of the compared algorithms with different user–server configurations (M, N).

comparison at $(M, N) = (40, 20)$, the nearest server, random offload, and random server methods show latency increases of approximately 29.73, 70.98, and 120.76 ms above the global method, which corresponds to 23.84, 65.08, and 114.86 ms higher latency than EEP. These results highlight the efficiency of EEP under different (M, N) configurations.

VI. Conclusion

In this article, we have presented OPLA, a novel cross-layer optimization framework designed to jointly manage latency and concurrency in serverless edge computing. By incorporating both application-layer concurrency and physical-layer resource constraints, OPLA addresses a critical gap in existing literature. The formulated problem is a challenging MINLP and is proven to be *NP*-complete under realistic offloading conditions. To tackle this intractability, we proposed an efficient EEP method. This approach leverages hidden convex structures via SCAs, deterministic rounding, and a presolve–postsolve routine to approximate the optimal solution with practical efficiency. Extensive numerical evaluations under diverse network conditions demonstrate that EEP achieves near-optimal performance, maintaining a relative difference below 6% compared to the global method, while significantly outperforming conventional baselines in terms of latency and resource utilization. The proposed solution is scalable, parallelizable, and robust across workload intensities and resource availability, making it a compelling candidate for real-world deployment in latency-sensitive serverless edge environments.

Although the exponential-cone formulation compactly captures key nonlinearities, it requires specialized solvers. Future work will explore simpler reformulations, first-order methods, and distributed approaches such as alternating direction method of multipliers (ADMM) for improved scalability.

Appendix A
Proof of Theorem 1

Consider a representative case of OPLA, where certain UEs are connected to ESs, i.e., some entries of $\boldsymbol{A}$ are set to 1. As discussed in Section II-B, let $\boldsymbol{X}' = \mathrm{vec}(\boldsymbol{X})$. Then, constraint (7) can be reformulated as $\boldsymbol{I}_N \otimes \boldsymbol{B}_{M\times T}\boldsymbol{X}'_{NT\times 1} \geqslant \mathrm{vec}\left(\boldsymbol{A}_{M\times N}\right)$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

where $\boldsymbol{I}_N$ is an $N \times N$ identity matrix. This highlights the block-diagonal structure of the constraint

$$\begin{bmatrix} \boldsymbol{B}_{M\times T} & 0 & \cdots & 0 \\ 0 & \boldsymbol{B}_{M\times T} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \boldsymbol{B}_{M\times T} \end{bmatrix} \boldsymbol{X}'_{NT\times 1} \geqslant \text{vec}(\boldsymbol{A}_{M\times N}). \tag{A.1}$$

We see that (A.1) represents a system of set cover constraints, and the objective $\sum_{n\in\mathcal{N}} \sum_{t\in\mathcal{T}} x_{nt}$ corresponds to the classical set cover objective [57]. Thus, optimizing this sum subject to (A.1) and the binary constraint (14e) reduces to an instance of the set cover problem. According to [57, Th. 8.16], set cover is *NP*-complete. Hence, this case of OPLA is *NP*-complete, implying that the general OPLA problem is also *NP*-complete by reduction.

REFERENCES

[1] Y. Zhang, *Mobile Edge Computing*. Cham, Switzerland: Springer, 2022.

[2] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.

[3] B. Kar, W. Yahya, Y.-D. Lin, and A. Ali, "Offloading using traditional optimization and machine learning in federated cloud–edge–fog systems: A survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1199–1226, Jan. 2023.

[4] M. Tari, M. Ghobaei-Arani, J. Pouramini, and M. Ghorbian, "Auto-scaling mechanisms in serverless computing: A comprehensive review," *Comput. Sci. Rev.*, vol. 53, Aug. 2024, Art. no. 100650.

[5] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski, "The rise of serverless computing," *Commun. ACM*, vol. 62, no. 12, pp. 44–54, Nov. 2019.

[6] CNCF Serverless Working Group. (Feb. 2018). *CNCF Serverless Whitepaper, Version 1.0*. Accessed: Aug. 31, 2025. [Online]. Available: https://github.com/cncf/wg-serverless/blob/master/whitepapers/serverless-overview/cncf_serverless_whitepaper_v1.0.pdf

[7] S. R. Chaudhry, A. Palade, A. Kazmi, and S. Clarke, "Improved QoS at the edge using serverless computing to deploy virtual network functions," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 10673–10683, Oct. 2020.

[8] Amazon Web Services. (2025). *AWS Lambda Developer Guide*. Accessed: Aug. 31, 2025. [Online]. Available: https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html

[9] Amazon Web Services. (2025). *AWS Lambda*. Accessed: Aug. 31, 2025. [Online]. Available: https://aws.amazon.com/lambda/?nc1=h_ls

[10] L. Chen, "A hybrid concurrency control protocol for TSFS," University of California, Tech. Rep. UCB/EECS-2020-196, Dec. 2020.

[11] T. Q. Dinh, J. Tang, Q. D. La, and T. Q. S. Quek, "Offloading in mobile edge computing: Task allocation and computational frequency scaling," *IEEE Trans. Commun.*, vol. 65, no. 8, pp. 3571–3584, Aug. 2017.

[12] T. Q. Dinh, B. Liang, T. Q. S. Quek, and H. Shin, "Online resource procurement and allocation in a hybrid edge-cloud computing system," *IEEE Trans. Wireless Commun.*, vol. 19, no. 3, pp. 2137–2149, Mar. 2020.

[13] L. Pan, L. Wang, S. Chen, and F. Liu, "Retention-aware container caching for serverless edge computing," in *Proc. IEEE Conf. Comput. Commun.*, May 2022, pp. 1069–1078.

[14] Y. Li, D. Zeng, L. Gu, M. Ou, and Q. Chen, "On efficient zygote container planning toward fast function startup in serverless edge cloud," in *Proc. IEEE Conf. Comput. Commun.*, May 2023, pp. 1–9.

[15] S. Hu, Z. Qu, B. Tang, B. Ye, G. Li, and W. Shi, "Joint service request scheduling and container retention in serverless edge computing for vehicle-infrastructure collaboration," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 6508–6521, Jun. 2024.

[16] X. Shang, Y. Mao, Y. Liu, Y. Huang, Z. Liu, and Y. Yang, "Online container scheduling for data-intensive applications in serverless edge computing," in *Proc. IEEE Conf. Comput. Commun.*, May 2023, pp. 1–10.

[17] J. Li et al., "Mobility-aware utility maximization in digital twin-enabled serverless edge computing," *IEEE Trans. Comput.*, vol. 73, no. 7, pp. 1837–1851, Jul. 2024.

[18] M. Li and Z. Wang, "Cost-AoI aware task scheduling in industrial IoT based on serverless edge computing," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2024, pp. 1–6.

[19] N. H. Nguyen, V.-D. Nguyen, A. T. Nguyen, N. V. Thieu, H. N. Nguyen, and S. Chatzinotas, "Deadline-aware joint task scheduling and offloading in mobile-edge computing systems," *IEEE Internet Things J.*, vol. 11, no. 20, pp. 33282–33295, Oct. 2024.

[20] D. Van Huynh, V.-D. Nguyen, S. Chatzinotas, S. R. Khosravirad, H. V. Poor, and T. Q. Duong, "Joint communication and computation offloading for ultra-reliable and low-latency with multi-tier computing," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 521–537, Feb. 2023.

[21] T. T. Bui et al., "Task offloading optimization for UAV-aided NOMA networks with coexistence of near-field and far-field communications," *IEEE Trans. Green Commun. Netw.*, vol. 9, no. 1, pp. 327–337, Mar. 2025.

[22] S. Yu, R. Langar, X. Fu, L. Wang, and Z. Han, "Computation offloading with data caching enhancement for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 11098–11112, Nov. 2018.

[23] S. Pan, H. Zhao, Z. Cai, D. Li, R. Ma, and H. Guan, "Sustainable serverless computing with cold-start optimization and automatic workflow resource scheduling," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 329–340, May 2023.

[24] R. Xie, Q. Tang, S. Qiao, H. Zhu, F. R. Yu, and T. Huang, "When serverless computing meets edge computing: Architecture, challenges, and open issues," *IEEE Wireless Commun.*, vol. 28, no. 5, pp. 126–133, Oct. 2021.

[25] J. Zhang et al., "Energy-latency tradeoff for energy-aware offloading in mobile edge computing networks," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2633–2645, Aug. 2018.

[26] Y. Sun, C. Song, S. Yu, Y. Liu, H. Pan, and P. Zeng, "Energy-efficient task offloading based on differential evolution in edge computing system with energy harvesting," *IEEE Access*, vol. 9, pp. 16383–16391, 2021.

[27] F. Tütüncüoğlu and G. Dán, "Joint resource management and pricing for task offloading in serverless edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7438–7452, Jun. 2024.

[28] R. Lin et al., "Distributed optimization for computation offloading in edge computing," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 8179–8194, Dec. 2020.

[29] T. Zhou, D. Qin, X. Nie, X. Li, N. Jiang, and C. Li, "Joint computation offloading and resource optimization for minimizing network-wide energy consumption in ultradense MEC networks," *IEEE Syst. J.*, vol. 18, no. 2, pp. 1115–1126, Jun. 2024.

[30] D. V. Huynh, V. Nguyen, O. A. Dobre, S. R. Khosravirad, and T. Q. Duong, "Adaptive service placement, task offloading and bandwidth allocation in task-oriented URLLC edge networks," in *Proc. IEEE ICC*, Jun. 2023, pp. 5755–5760.

[31] H. Govind and H. González–Vélez, "Benchmarking serverless workloads on Kubernetes," in *Proc. IEEE CCGrid*, Aug. 2021, pp. 704–712.

[32] R. B. Roy, T. Patel, R. Liew, Y. Babuji, R. Chard, and D. Tiwari, "ProPack: Executing concurrent serverless functions faster and cheaper," in *Proc. HPDC*, Aug. 2023, pp. 211–224.

[33] P. Gackstatter, P. A. Frangoudis, and S. Dustdar, "Pushing serverless to the edge with WebAssembly runtimes," in *Proc. IEEE CCGrid*, Jul. 2022, pp. 140–149.

[34] N. Mahmoudi and H. Khazaei, "Performance modeling of metric-based serverless computing platforms," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1899–1910, Apr. 2023.

[35] L. Schuler, S. Jamil, and N. Kühl, "AI-based resource allocation: Reinforcement learning for adaptive auto-scaling in serverless environments," in *Proc. IEEE CCGrid*, Aug. 2021, pp. 804–811.

[36] B. R. Marks and G. P. Wright, "A general inner approximation algorithm for nonconvex mathematical programs," *Oper. Res.*, vol. 26, no. 4, pp. 681–683, Jul. 1978.

[37] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[38] S. T. Le, K. Schuh, R. Dischler, F. Buchali, L. Schmalen, and H. Buelow, "Beyond 400 Gb/s direct detection over 80 km for data center interconnect applications," *J. Lightw. Technol.*, vol. 38, no. 2, pp. 538–545, Jan. 2020.

[39] N. Sharma, "Parallel serverless workloads in OpenWhisk on Kubernetes," Master's thesis, National College of Ireland, Dublin, Ireland, 2022.

[40] V. Farhadi et al., "Service placement and request scheduling for data-intensive applications in edge clouds," *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 779–792, Apr. 2021.

[41] B. Gao, Z. Zhou, F. Liu, and F. Xu, "Winning at the starting line: Joint network selection and service placement for mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1459–1467.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.

[42] N. Xuan Tung, T. Van Chien, D. Thai Hoang, and W. J. Hwang, "Jointly optimizing power allocation and device association for robust IoT networks under infeasible circumstances," *IEEE Trans. Netw. Service Manage.*, vol. 22, no. 2, pp. 1372–1390, Apr. 2025.
[43] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, vol. 174. San Francisco, CA, USA: Freeman, 1979.
[44] T. Berthold, "Heuristic algorithms in global MINLP solvers," Doctoral dissertation, Technische Universität Berlin, Nov. 2014.
[45] K. Deb, *Multi-objective Optimization Using Evolutionary Algorithms*. Hoboken, NJ, USA: Wiley, Jun. 2001.
[46] T. Lattimore and C. Szepesvári, *Bandit Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2020.
[47] S. S. Rao, *Engineering Optimization: Theory and Practice*. Hoboken, NJ, USA: Wiley, 2019.
[48] H. Nguyen-Kha, V. Nguyen Ha, E. Lagunas, S. Chatzinotas, and J. Grotz, "Joint two-tier user association and resource management for integrated satellite-terrestrial networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 11, pp. 16648–16665, Nov. 2024.
[49] L. Wittenberg, *Data Structures and Algorithms in C++: Pocket Primer*. Mercury Learning and Information, 2017.
[50] A. A. Nasir, H. D. Tuan, H. H. Nguyen, M. Debbah, and H. V. Poor, "Resource allocation and beamforming design in the short blocklength regime for URLLC," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 1321–1335, Feb. 2021.
[51] E. Peltonen et al., "The many faces of edge intelligence," *IEEE Access*, vol. 10, pp. 104769–104782, 2022.
[52] R. Bruschi, F. Davoli, P. Lago, C. Lombardo, and J. F. Pajo, "Personal services placement and low-latency migration in edge computing environments," in *Proc. IEEE NFV-SDN*, May 2018, pp. 1–6.
[53] L. Tang and H. Hu, "Computation offloading and resource allocation for the Internet of Things in energy-constrained MEC-enabled HetNets," *IEEE Access*, vol. 8, pp. 47509–47521, 2020.
[54] R. K. Jain, D.-M. W. Chiu, and W. R. Hawe, *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer System*, Standard DEC-TR-301, 1984.
[55] M. Hui, J. Chen, Y. Zhou, B. He, K. Wu, and L. Yang, "Server deployment and load balancing in stochastic mobile edge computing networks," *IEEE Commun. Lett.*, vol. 26, no. 5, pp. 1194–1198, May 2022.
[56] H. S. Ryoo and N. V. Sahinidis, "A branch-and-reduce approach to global optimization," *J. Global Optim.*, vol. 8, no. 2, pp. 107–138, Mar. 1996.
[57] J. Kleinberg and E. Tardos, *Algorithm Design: Pearson New International Edition*. London, U.K.: Pearson, 2005.



Minh-Tuong Nguyen is currently pursuing the joint degree with Collaborative Doctoral Research Degree Program from VinUniversity, Hanoi, Vietnam, and the University of Technology Sydney, Ultimo, NSW, Australia.

His research interests include mobile edge computing, physical-layer security, secrecy outage probability, computational algebraic geometry, and resource allocation in wireless communications.



Quang-Trung Luu (Member, IEEE) received the Ph.D. degree from the CentraleSupélec, Université Paris-Saclay, Gif-sur-Yvette, France, in 2022.

He was an Assistant Professor with the School of Electrical and Electronic Engineering, Hanoi University of Science and Technology, Hanoi, Vietnam, from 2023 to 2025, and a Post-Doctoral Researcher with the University of Avignon, Avignon, France, and LAAS–CNRS, Toulouse, France, from 2021 to 2023. He is currently an Associate Professor (maître de conférences) with the Laboratory of Signals and Systems (L2S), CentraleSupélec, Université Paris-Saclay. His research interests include resource management and orchestration in next-generation communication networks (5G and beyond), leveraging mathematical optimization, and artificial intelligence.

Dr. Luu serves as a Co-Coordinator for the Industry Communities of the Next-Generation Networking and Internet Technical Committee (NGNI-TC) of the IEEE Communications Society.



Vo Phi Son received the B.S. degree in communications–electronics engineering technology from Vietnam Aviation Academy, Ho Chi Minh City, Vietnam, in 2012, and the M.S. degree in telecommunications engineering from Ho Chi Minh City University of Technology, Ho Chi Minh City, in 2016. He is currently pursuing the Ph.D. degree with the College of Engineering and Computer Science, VinUniversity, Hanoi, Vietnam.

His current research interests include mobile edge computing, integrated sensing and communications, convex optimization, and machine learning for wireless communications.



Le-Nam Tran (Senior Member, IEEE) received the B.S. degree in electrical engineering from Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam, in 2003, and the M.S. and Ph.D. degrees in radio engineering from Kyung Hee University, Seoul, South Korea, in 2006 and 2009, respectively.

He was a Lecturer with the Department of Electronic Engineering, Maynooth University, Maynooth, Ireland. From 2010 to 2014, he held post-doctoral positions at the Signal Processing Laboratory, ACCESS Linnaeus Centre, KTH Royal Institute of Technology, Stockholm, Sweden, and the Centre for Wireless Communications, University of Oulu, Oulu, Finland. He is currently an Associate Professor with the School of Electrical and Electronic Engineering, University College Dublin, Dublin, Ireland. His research interests include applications of optimization techniques for wireless communications design. Some recent topics include energy-efficient communications, cell-free massive MIMO, intelligent surfaces, integrated sensing and communications, physical-layer security, and cloud radio access networks.

Dr. Tran was a recipient of the Career Development Award from Science Foundation Ireland in 2018. He was a co-recipient of the IEEE GLOBECOM 2021 Best Paper Award and the IEEE PIMRC 2020 Best Student Experimental Paper Award. He has served on the technical program committees for several major IEEE conferences. He was the Symposium Co-Chair of the Cognitive Computing and Networking Symposium of the International Conference on Computing, Networking and Communication (ICNC 2016) and the Co-Chair of the Workshop on Scalable Massive MIMO Technologies for Beyond 5G at IEEE ICC 2020. He is an Associate Editor of IEEE TRANSACTIONS ON COMMUNICATIONS, IEEE COMMUNICATIONS LETTERS, and *EURASIP Journal on Wireless Communications and Networking*.



Van-Dinh Nguyen (Senior Member, IEEE) was an Assistant Professor with VinUniversity, Hanoi, Vietnam, a Research Associate with the SnT University of Luxembourg, a Post-Doctoral Researcher and a Lecturer with Soongsil University, Seoul, South Korea, a Post-Doctoral Visiting Scholar with the University of Technology Sydney, Ultimo, NSW, Australia, and a Ph.D. Visiting Scholar with Queen's University Belfast. He has been an Assistant Professor with the School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland, since 2025. He has authored or co-authored over 90 papers published in international journals and conference proceedings. His current research activity is focused on open RAN, wireless sensing, edge/fog computing, and AI/ML solutions for wireless communications.

Dr. Nguyen received the four Best Conference Paper Awards and the four Exemplary Editor Awards from IEEE COMMUNICATIONS LETTERS and IEEE OPEN JOURNAL OF THE COMMUNICATIONS SOCIETY. He has served as a reviewer for many top-tier international journals on wireless communications and as a technical program committee member for several flagship international conferences in related fields. He is an Editor of IEEE OPEN JOURNAL OF THE COMMUNICATIONS SOCIETY and IEEE SYSTEMS JOURNAL and a Senior Editor of IEEE COMMUNICATIONS LETTERS.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:21:56 UTC from IEEE Xplore. Restrictions apply.